

Table of Contents

[TestLibraryDefTopic](#)

[tlCommonParams.9.01](#)

[tlCommonParams.9.02](#)

[tlCommonParams.9.03](#)

[tlCommonParams.9.04](#)

[tlCommonParams.9.05](#)

[tlCommonParams.9.06](#)

[tlCommonParams.9.07](#)

[tlCommonParams.9.08](#)

[tlCommonParams.9.09](#)

[tlCommonParams.9.10](#)

[tlCommonParams.9.11](#)

[tlCommonParams.9.12](#)

[tlCommonParams.9.13](#)

[tlCommonParams.9.14](#)

[tlCommonParams.9.15](#)

[tlCommonParams.9.16](#)

[tlCommonParams.9.17](#)

[tlCommonParams.9.18](#)

[tlCommonParams.9.19](#)

[tlCommonParams.9.20](#)

[tlCommonParams.9.21](#)

[tlCommonParams.9.22](#)

[tlCommonParams.9.23](#)

[tlCommonParams.9.24](#)

[tlCommonParams.9.25](#)

[tlCommonParams.9.26](#)

[tlCommonParams.9.27](#)

[tlCommonParams.9.28](#)

[tlCommonParams.9.29](#)
[tlCommonParams.9.30](#)
[tlCommonParams.9.31](#)
[tlCommonParams.9.32](#)
[tlCommonParams.9.33](#)
[tlCommonParams.9.34](#)
[tlCommonParams.9.35](#)
[tlCommonParams.9.36](#)
[tlCommonParams.9.37](#)
[tlCommonParams.9.38](#)
[tlDcviPowerSupply.3.01](#)
[tlDcviPowerSupply.3.02](#)
[tlDcviPowerSupply.3.03](#)
[tlDcviPowerSupply.3.04](#)
[tlDcviPowerSupply.3.05](#)
[tlDcviPowerSupply.3.06](#)
[tlDcviPowerSupply.3.07](#)
[tlDcviPowerSupply.3.08](#)
[tlDcviPowerSupply.3.09](#)
[tlDcviPowerSupply.3.10](#)
[tlDcviPowerSupply.3.11](#)
[tlDcviPowerSupply.3.12](#)
[tlDcviPowerSupply.3.13](#)
[tlDcviPowerSupply.3.14](#)
[tlDcvsPowerSupply.4.01](#)
[tlDcvsPowerSupply.4.02](#)
[tlDcvsPowerSupply.4.03](#)
[tlDcvsPowerSupply.4.04](#)
[tlDcvsPowerSupply.4.05](#)
[tlDcvsPowerSupply.4.06](#)
[tlDcvsPowerSupply.4.07](#)
[tlDcvsPowerSupply.4.08](#)

[tlDcvsPowerSupply.4.09](#)

[tlDcvsPowerSupply.4.10](#)

[tlDcvsPowerSupply.4.11](#)

[tlDcvsPowerSupply.4.12](#)

[tlDcvsPowerSupply.4.13](#)

[tlEmpty](#)

[tlFunctional.6.01](#)

[tlFunctional.6.02](#)

[tlFunctional.6.03](#)

[tlFunctional.6.04](#)

[tlFunctional.6.05](#)

[tlFunctional.6.06](#)

[tlFunctional.6.07](#)

[tlFunctional.6.08](#)

[tlFunctional.6.09](#)

[tlFunctional.6.10](#)

[tlMtoMemory.7.01](#)

[tlMtoMemory.7.02](#)

[tlMtoMemory.7.03](#)

[tlMtoMemory.7.04](#)

[tlMtoMemory.7.05](#)

[tlMtoMemory.7.06](#)

[tlMtoMemory.7.07](#)

[tlMtoMemory.7.08](#)

[tlMtoMemory.7.09](#)

[tlMtoMemory.7.10](#)

[tlMtoMemory.7.11](#)

[tlMtoMemory.7.12](#)

[tlMtoMemory.7.13](#)

[tlMtoMemory.7.14](#)

[tlMtoMemory.7.15](#)

[tlMtoMemory.7.16](#)

[tlMtoMemory.7.17](#)

[tlMtoMemory.7.18](#)

[tlMtoMemory.7.19](#)

[tlMtoMemory.7.20](#)

[tlMtoMemory.7.21](#)

[tlMtoMemory.7.22](#)

[tlMtoMemory.7.23](#)

[tlMtoMemory.7.24](#)

[tlMtoMemory.7.25](#)

[tlMtoMemory.7.26](#)

[tlMtoMemory.7.27](#)

[tlMtoMemory.7.28](#)

[tlMtoMemory.7.29](#)

[tlMtoMemory.7.30](#)

[tlMtoMemory.7.31](#)

[tlMtoMemory.7.32](#)

[tlMtoMemory.7.33](#)

[tlMtoMemory.7.34](#)

[tlMtoMemory.7.35](#)

[tlPinPmu.8.01](#)

[tlPinPmu.8.02](#)

[tlPinPmu.8.03](#)

[tlPinPmu.8.04](#)

[tlPinPmu.8.05](#)

[tlPinPmu.8.06](#)

[tlPinPmu.8.07](#)

[tlPinPmu.8.08](#)

[tlPinPmu.8.09](#)

[tlPinPmu.8.10](#)

[tlPinPmu.8.11](#)

[tlPinPmu.8.12](#)

[tlPinPmu.8.13](#)

[tlPinPmu.8.14](#)[tlPinPmu.8.15](#)[tlPinPmu.8.16](#)[tlPinPmu.8.17](#)[tlPinPmu.8.18](#)[tlPinPmu.8.19](#)[tlPinPmu.8.20](#)[tlPinPmu.8.21](#)[tlPinPmu.8.22](#)[tlTopics.2.01](#)[tlTopics.2.02](#)[tlTopics.2.03](#)[tlTopics.2.04](#)[tlTopics.2.05](#)[tlTopics.2.06](#)[tlTopics.2.07](#)[tlTopics.2.08](#)[tlTopics.2.09](#)[tlTopics.2.10](#)[tlTopics.2.11](#)[tlTopics.2.12](#)[tlTopics.2.13](#)[tlTopics.2.14](#)

TestLibraryDefTopic



About VBT Test Library

About VBT Test Library

The Teradyne-supplied VBT test library is a set of basic, commonly used tests that can be the basis for user-created tests in a job.

Each VBT test contains the logic to perform the testing actions, but not the actual values to be used in testing. You supply these values as parameters to the VBT test. A test instance is a VBT test that has had particular data values applied to it to perform a particular test. The test instance name associates this test instance with this set of parameters.

For example, "Functional_T" is a Teradyne-supplied VBT test used to perform a device functional test. On a Test Instances sheet, you create a test instance named "MyFct" and indicate that it is based on the Functional_T VBT test. For more information, see [Creating a Test Instance](#).

You then specify its parameters, including what Time Sets and Pin Levels sheets to use, what patterns to apply to the DUT, and so on. To help with entering parameter values, the VBT tests have instance editors. An instance editor is a TDE tool that clearly labels each parameter and, where appropriate, provides a list of valid values, or other windows that facilitate entering the parameter value. You bring up the instance editor by selecting the test instance name, right-clicking, and selecting [Edit](#). For more information, see [Using an Instance Editor](#).

The tests in the VBT test library correspond to instruments that are used for testing:

- [DCVI Power Supply VBT Test](#)
- [DCVS Power Supply VBT Test](#)
- [Functional VBT Test](#)
- [MTO Memory VBT Test](#)
- [Pin PMU VBT Test](#)

There is also an Empty VBT test that can be used to set common parameters. See [Empty VBT Test](#).

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.01

<	>
-----------------	-----------------

Common Parameters

Common Parameters

Several tests in the Teradyne VBT test library use the parameters described in this section. These parameters' behavior is not instrument-specific and is therefore not included with the instrument-related tests.

The following types of parameters are included:

- | | |
|---|---------------------------------|
| • | Flag Match Parameters |
| • | Hold State Pattern Parameters |
| • | Interpose Functions Parameters |
| • | Pin States Parameters |
| • | Precondition Pattern Parameters |
| • | Timing and Levels Tab |
| • | Limits Tab |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.02

<

>

Common Parameters : Flag Match Parameters

Flag Match Parameters

Flag Match parameters let you specify a function that is be called when the executing pattern reaches a certain state. They include the following:

- WaitFlagA/B/C/D
- MatchAllSites
- PatFlagF and PatFlagFArgs

The PatFlagF function can perform any action you want. One frequent use is known as “VDD bump.” At a specific point in pattern execution, under control of the pattern, you must change the power supply voltage while the pattern is running and then to continue the pattern. This action can be performed from the PatFlagF function.

Using the Flag Match parameters usually involves these steps (details are given below):

- The specified pattern uses one or more CPU flags (D, C, B, A) to construct an infinite loop. For example, set CPU flag A to True, and then loop while CPU flag A is True. The pattern can also set other CPU flags, to indicate that a certain state has been reached.
- The test instance specifies WaitFlag parameters, which have a expected value of waitLo, waitHi, or waitOff (ignore) for each of the four CPU flags. The default is waitOff for all four flags. For Flag Match to be in effect, at least one of the wait flags must have a nondefault value of waitHi or waitOff.

- The test starts the pattern and waits for the WaitFlag conditions to be met. Once they are met, the PatFlagF function is executed, with any arguments. The test then inverts the pattern’s CPU flags, causing an exit from the loop for continued pattern execution.

Each pattern burst in the pattern list is required to result in at least one flag match on each site. If the pattern halts and passes without a match, a run-time error is returned.

Execution Sequence

If a test instance has specified flag match parameters, this sequence of actions is performed for each pattern in the pattern list:

1.

Start the pattern.
2.

While the pattern is running on any site, periodically check the CPU flags for a match with WaitFlags.
3.

The next action depends on whether pattern threading is off or on:

a.

If PatThreading is False, or if PatThreading is True and MatchAllSites is True, keep checking until all sites match. Then call PatFlagF, and then invert the matched flags on all sites.

a.

If PatThreading is True and MatchAllSites is False, when one site matches, call PatFlagF, then invert the flag for that site. Continue checking the remaining sites. When each site matches, invert the flags for that site. Note that PatFlagF is called only when the first site matches, not for subsequent sites.
4.

A flag match must occur for each site. If the pattern passes and a flag match has not been found for a site, a run-time error is returned.

These actions occur for each pattern burst in the pattern list. This means that PatFlagF is called once for each pattern burst in the list.

<

>

tlCommonParams.9.03

<

>

Common Parameters : [Flag Match Parameters](#) : [WaitFlagA/B/C/D](#)

[WaitFlagA/B/C/D](#)

The CPU flag conditions that must be met before the function is executed.

[Valid Values](#)

Values are available from the drop-down list.

For each flag ([WaitFlagA](#), [WaitFlagB](#), [WaitFlagC](#), and [WaitFlagD](#)) the values are:

waitLo	The corresponding CPU flag must be False.
waitHi	The corresponding CPU flag must be True.
waitOff	Ignore the corresponding CPU flag. Default.

[Usage](#)

The default is [waitOff](#) for all four flags. For the [PatFlagF](#) to be executed, at least one of the flag fields must have its value set to [waitLo](#) or [waitHi](#).

[Related Topics](#)

- [Flag Match Parameters](#)
- [MatchAllSites](#)
- [PatFlagF and PatFlagFArgs](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlCommonParams.9.04

<

>

Common Parameters : [Flag Match Parameters](#) : MatchAllSites

MatchAllSites

Whether all sites or only one site must match [WaitFlags](#) for the `PatFlagF` function to be executed.

Valid Values

The values are available from the drop-down list:

True	All sites must match the WaitFlags .
False	Only one site must match the WaitFlags . Default.

Usage

This parameter is used only if `PatThreading` is True.

When pattern threading is turned on, patterns are executed independently on each site, and the sites may match the CPU flags independently. The `MatchAllSites` parameter lets you specify whether the `PatFlagF` function is to be executed after any one site matches or after all sites match. If pattern threading is turned off, this parameter is ignored, because all sites must match.

Related Topics

- [Flag Match Parameters](#)
- [WaitFlagA/B/C/D](#)
- [PatFlagF and PatFlagFArgs](#)
- [Pattern Threading](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.05

	
---	---

[Common Parameters](#) : [Flag Match Parameters](#) : PatFlagF and PatFlagFArgs

PatFlagF and PatFlagFArgs

The function to execute, plus arguments, when the flags match.

Valid Values

The drop-down list contains the names of the available VBT functions that are in modules beginning with VBT. Select a function from the list.

If the function requires parameters, enter them as a comma-separated list in the PatFlagFArgs field.

Each comma-delimited item is treated as a string. Do not enquote them.

Usage

The PatFlagF function is identical to an interpose function, with the same calling convention, parameter string, and location.

Related Topics

- [Flag Match Parameters](#)
- [Interpose Functions Parameters](#)
- [WaitFlagA/B/C/D](#)
- [MatchAllSites](#)

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.06

<

>

Common Parameters : Hold State Pattern Parameters

Hold State Pattern Parameters

A hold-state pattern is a single pattern file or pattern set that is to be started before a parametric measurement is made and stopped after the measurement is made. The hold-state pattern permits parametric measurements to be executed dynamically, while the pattern generator is running. Standard use of HoldStatePat is as follows:

- The specified pattern uses one or more CPU flags (A, B, C, D) to construct an infinite loop.
- The test instance specifies WaitFlag parameters, which have a expected value of waitLo, waitHi, or waitOff (ignore) for each of the four CPU flags.
- The test starts the pattern and waits for the WaitFlags conditions to be met. Once they are met, the parametric measurement is made. The test then either halts the pattern or causes an exit from the loop for continued execution.

If the pattern does not use a loop or if it loops using a construction other than the CPU flags, or if the test uses the default WaitFlags (all waitOff), then the standard use does not apply. Note that concurrent test is disabled when executing a hold-state pattern in a VBT test instance. For more information, see:

- Standard Use of Hold-State Patterns
- Other Uses of Hold-State Patterns

Hold State Parameters Quick Reference

The following parameters are used to control the hold-state pattern:

HoldStatePat	Pattern or pattern set to be run.
HspStartLabel	Label at which execution is to start.
HspStopLabel	The label at which execution is to stop.
HspCheckPatGen	Whether to check the pattern generator for pass/fail status and to pass the status to the flow controller.
HspResumePat	Whether to resume pattern execution after the parametric measurement.
HspDisableAlarmCheck	Whether to disable instrument alarms that occur during execution of the hold-state pattern.
WaitFlagA/B/C/D	CPU flag conditions that must be met before the parametric measurement is made.
PatternTimeout	Time-out period, in seconds.

See also [Validation of Patterns and Labels](#).

tlCommonParams.9.07

<	>
-----------------	-----------------

Common Parameters : [Hold State Pattern Parameters](#) : Standard Use of Hold-State Patterns

Standard Use of Hold-State Patterns

The standard use assumes that the [HoldStatePat](#) pattern contains a loop controlled by CPU flags and that the test instance's [WaitFlag](#) parameters have at least one nondefault value (that is, other than [waitOff](#)).

If the pattern does not use a loop or if it loops using a construction other than the CPU flags, or if the test instance uses the default [WaitFlags](#) (all [waitOff](#)), then the standard use does not apply. See [Other Uses of Hold-State Patterns](#).

Sequence

In standard use, the sequence of test actions is as follows:

- | | |
|----|--|
| 1. | Clear any CPU flags in the pattern. |
| 2. | Set the pattern time-out to PatternTimeout . Time-out is always enabled. |
| 3. | Start the pattern from the HspStartLabel , if one is specified. Otherwise, start from the beginning of the pattern. |
| 4. | Wait until the WaitFlag conditions are met in the pattern. If the time-out expires before the conditions are met, set the test status to Fail. |
| 5. | If HspCheckPatGen is 1 (enable), check the pass/fail status of the pattern generator (before the parametric measurement is made) and return the status to the flow controller. |
| 6. | Make the parametric measurement. |

7. [After the measurement:](#)
- a. [If](#) HspResumePat is True, set the CPU flags in the pattern to a condition opposite that needed to set up the loop. This causes the pattern to exit from its loop (assuming the loop is constructed using CPU flags) and permits pattern execution to continue. The time-out for this portion of the execution is reset to PatternTimeout.

b. [If](#) HspResumePat is False and there is no stop label, halt the pattern.

c. [If](#) HspResumePat is False but there is a stop label, wait for pattern execution to halt by reaching the stop label or by timing out. (The CPU flags are not reset, so there must be some other mechanism to exit from the loop.) Time-out is not reset, but the time-out that was set before the parametric measurement is still in effect.
8. [If](#) HspCheckPatGen is 1 (enable), check the pass/fail status of the pattern generator again and return the status to the flow controller.

tlCommonParams.9.08

<

>

[Common Parameters](#) : [Hold State Pattern Parameters](#) : Other Uses of Hold-State Patterns

Other Uses of Hold-State Patterns

The standard use assumes that the HoldStatePat pattern contains a loop controlled by CPU flags and that the test instance's WaitFlag parameters have at least one nondefault value (that is, other than waitOff). See [Standard Use of Hold-State Patterns](#).

If either of these conditions is not met, the HoldStatePat argument can still be used, but differences in its behavior must be noted.

The possible combinations of pattern looping and WaitFlags are as follows.

Pattern Loops, Default WaitFlags

The HoldStatePat pattern can be used for dynamic conditioning, although the default WaitFlag values mean no pattern control from the test. The pattern must use jump opcodes to set and control the loop.

Because the WaitFlags do not set flag conditions, the pattern runs until it is halted in one of these ways:

- By a stop label (if specified)
- By the end of the pattern
- By a halt command from the test code
- By the expiration of a 30 second time-out (any WaitTime value is ignored)

If HspCheckPatGen is 1 (enable), the pattern generator is checked only after the measurement (assuming the pattern is not halted), not before.

Pattern Does Not Loop, Nondefault WaitFlags

The HoldStatePat pattern can be used for static conditioning. The measurement will not occur while the PatGen is running. This use assumes that the pattern uses CPU flag values to indicate that certain points within the pattern have been reached. When these points are reached, the test's WaitFlag conditions are met, and the parametric measurement can then be made.

Pattern Does Not Loop, Default WaitFlags

The HoldStatePat pattern can be used for static conditioning. There is no interaction between the pattern and the test.

This use is not recommended. Use the PreCondPat pattern for this kind of conditioning. See Precondition Pattern Parameters.

<		>	
		2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.09

<	>
-----------------	-----------------

Common Parameters : [Hold State Pattern Parameters](#) : HoldStatePat

HoldStatePat

The pattern or pattern set to be run.

Valid Values

To select a pattern, click the ellipsis button (...) to bring up the Pattern Selector tool, which lets you select a pattern file or pattern set. You can also type in a pattern file pathname or pattern set name.

Related Topics

- | | |
|---|---|
| • | Pattern Selector |
| • | Validation of Patterns and Labels |
| • | Hold State Pattern Parameters |
| • | HspStartLabel |
| • | HspStopLabel |
| • | HspCheckPatGen |
| • | HspResumePat |
| • | HspDisableAlarmCheck |

- [WaitFlagA/B/C/D](#)

- [PatternTimeout](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.10

<	>
-----------------	-----------------

Common Parameters : [Hold State Pattern Parameters](#) : [HspStartLabel](#)

HspStartLabel

The label at which execution is to start. By default, execution starts at the beginning of the pattern.

Valid Values

Enter the name of a label. The label must be declared in the pattern file as a start label, global label, or global subr label.

Related Topics

- | | |
|---|---|
| • | Validation of Patterns and Labels |
| • | Hold State Pattern Parameters |
| • | HoldStatePat |
| • | HspStopLabel |
| • | HspCheckPatGen |
| • | HspResumePat |
| • | HspDisableAlarmCheck |
| • | WaitFlagA/B/C/D |

• PatternTimeout

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.11

<	>
-----------------	-----------------

Common Parameters : [Hold State Pattern Parameters](#) : [HspStopLabel](#)

HspStopLabel

The label at which execution is to stop.

Valid Values

Enter the name of a label. The label must be declared in the pattern file as a start label, global label, or global subr label.

If a stop label is specified, pattern execution runs to the stop label or until a time-out occurs.

Related Topics

- | | |
|---|---|
| • | Validation of Patterns and Labels |
| • | Hold State Pattern Parameters |
| • | HoldStatePat |
| • | HspStartLabel |
| • | HspCheckPatGen |
| • | HspResumePat |
| • | HspDisableAlarmCheck |
| • | WaitFlagA/B/C/D |

•	PatternTimeout
---	--------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.12

<

>

Common Parameters : [Hold State Pattern Parameters](#) : [HspCheckPatGen](#)

HspCheckPatGen

Whether to check the pattern generator for pass/fail status and to pass the status to the flow controller.

Valid Values

1	Enable. The status is checked twice: before the parametric measurement, and after the parametric measurement. If the pattern generator records a failure, the failure is returned to the flow controller, and the test fails.
0	Disable. The pattern generator is not checked. Even if the HoldStatePat pattern fails, the failure is not reflected in the test result that is returned to the flow controller. Default.

Usage

[HspCheckPatGen](#) lets you include a [HoldStatePat](#) failure (before or after the parametric measurement) as part of the test result as a whole. If [HspCheckPatGen](#) is 0 (disable), only the result of the parametric measurement affects the pass/fail test status. If it is 1 (enable), a pattern failure is also returned as a test failure. For example, after a successful parametric measurement, you can continue pattern execution beyond the CPU-flag loop and apply vectors to test a device’s state. If these vectors fail, the test as a whole is returned to the flow controller as a Fail.

Related Topics

- [Hold State Pattern Parameters](#)
- [HoldStatePat](#)

•	HspStartLabel
•	HspStopLabel
•	HspResumePat
•	HspDisableAlarmCheck
•	WaitFlagA/B/C/D
•	PatternTimeout

<	>
----------------------	----------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.13

<

>

Common Parameters : [Hold State Pattern Parameters](#) : [HspResumePat](#)

HspResumePat

Whether to resume pattern execution after the parametric measurement.

Valid Values

True	The test sets the CPU flags in the pattern to a condition that is the opposite of the condition needed to set up the loop. If the pattern loop is constructed properly, setting the CPU flags in this manner causes pattern execution to exit from the loop and continue with the rest of the pattern. Pattern execution continues until a halt or stop label is encountered or until PatternTimeout expires (if it is not 0).
False	The test halts the pattern execution (unless there is a stop label).

Usage

For usage details, see [Standard Use of Hold-State Patterns](#).

Related Topics

- [Hold State Pattern Parameters](#)
- [HoldStatePat](#)
- [HspStartLabel](#)
- [HspStopLabel](#)

- [HspCheckPatGen](#)
- [HspDisableAlarmCheck](#)
- [WaitFlagA/B/C/D](#)
- [PatternTimeout](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlCommonParams.9.14

<

>

Common Parameters : [Hold State Pattern Parameters](#) : [HspDisableAlarmCheck](#)

HspDisableAlarmCheck

Whether to disable instrument alarms that occur during execution of the hold-state pattern.

Valid Values

True	Disable the alarm window from being opened for the pattern burst.
False	Do not disable the alarm window.

Usage

By default, if a test bursts a pattern, the alarm window is opened and remains open until the pattern stops. This can cause a problem with unwanted alarms. For example, if the pattern is simply driving the clock pin, changes to the instrument will cause transient or temporary alarms. Because the alarm window is open, alarms are returned when the instrument measurement is made. To avoid reporting these alarms, set this parameter to True. This disables the alarm window from being opened for the hold-state pattern burst.

Related Topics

- [Hold State Pattern Parameters](#)
- [HoldStatePat](#)
- [HspStartLabel](#)
- [HspStopLabel](#)

- [HspCheckPatGen](#)
- [HspResumePat](#)
- [WaitFlagA/B/C/D](#)
- [PatternTimeout](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlCommonParams.9.15

<

>

Common Parameters : [Hold State Pattern Parameters](#) : [WaitFlagA/B/C/D](#)

WaitFlagA/B/C/D

The CPU flag conditions that must be met before the parametric measurement is made.

Valid Values

Values are available from the drop-down list.

For each flag (WaitFlagA, WaitFlagB, WaitFlagC, and WaitFlagD) the values are:

waitLo	The corresponding CPU flag must be False.
waitHi	The corresponding CPU flag must be True.
waitOff	Ignore the corresponding CPU flag. Default.

Usage

The default is waitOff for all four flags. For the HoldFlagF to be executed, at least one of the flag fields must have its value set to waitLo or waitHi.

Related Topics

- [Hold State Pattern Parameters](#)
- [HoldStatePat](#)
- [HspStartLabel](#)
- [HspStopLabel](#)

- [HspCheckPatGen](#)
- [HspResumePat](#)
- [HspDisableAlarmCheck](#)
- [PatternTimeout](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlCommonParams.9.16

<	>
-----------------	-----------------

Common Parameters : [Hold State Pattern Parameters](#) : PatternTimeout

PatternTimeout

The time-out period, in seconds.

Valid Values

Enter a value in the PatternTimeout box. Time-out is always enabled. The default time-out value is 30 seconds. A value of 0 means to halt execution immediately.

Usage

PatternTimeout is used as follows:

- Before the parametric measurement, PatternTimeout is the maximum time to wait for the WaitFlag conditions. If PatternTimeout expires before the WaitFlag conditions are met, pattern execution halts and the test is returned as a failure (if HspCheckPatGen is 1, which enables PatGen checking).
- After the parametric measurement, and if pattern execution continues beyond the CPU-flag loop (because HspResumePat is True), PatternTimeout is used again as the maximum time allowed for the pattern to execute. If PatternTimeout expires before a halt or stop label, pattern execution is halted and test execution continues.

Related Topics

- [Validation of Patterns and Labels](#)
- [Hold State Pattern Parameters](#)

•	HoldStatePat
•	HspStartLabel
•	HspStopLabel
•	HspCheckPatGen
•	HspResumePat
•	HspDisableAlarmCheck
•	WaitFlagA/B/C/D

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.17

<

>

Common Parameters : Interpose Functions Parameters

Interpose Functions Parameters

At certain defined points, a Teradyne-supplied VBT test executes a user-written function, if one exists. These user-written functions are called interpose functions. They let you modify the behavior of a standard test without modifying the code. An interpose function can make calls to the drivers to get and set driver values. You can also pass arguments to an interpose function. Most customization needs can be satisfied using interpose functions. Whenever possible, use interpose functions instead of creating customized versions of tests. The types of interpose functions, each called at a different point in test execution, are:

StartOfBodyF	Called just before the VBT test body.
PrePatF	Called before each pattern burst.
PreTestF	Called before the test is executed or the measurement is made.
PostTestF	Called after the test is executed or the measurement is made.
PostPatF	Called after each pattern burst.
EndOfBodyF	Called just after the VBT test body.
MeasF	Called by the DCVI Power Supply test if sites are serialized.

Some interpose functions are defined by their relationship to the three-part test structure, with a Prebody, Body, and Postbody. The main test function calls these three functions. See VBT Test Structure: Prebody, Body, Postbody. On filling in instance editor arguments, see Specifying Interpose Function Parameters. Interpose functions are called in different ways. The test code calls some explicitly. For the others, the main test function registers their names and arguments, so that they are available to driver

functions. The registered pattern interpose functions are implicitly called by the VBT statement that bursts the pattern. The other registered functions are called by an Exec VBT function that executes the interpose function identified by a registry key. You can examine the VBT test code to see how it calls the various interpose functions.

This section describes how to specify interpose functions and their arguments in the instance editor. For more on what goes into an interpose function, see [Writing Interpose Functions](#).

 Note:

In IG-XL V8.0, there is an issue with execution of interpose functions. If you have nested execution of Functional_T instances, the PrePatF, PostPatF, PreTestF, and PostTestF interpose functions may not execute correctly.

		
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.18

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : StartOfBodyF

StartOfBodyF

The interpose function, plus any arguments ([StartOfBodyFArgs](#)), that is called immediately before the body.

In the VBT test, the main test function calls the [StartOfBodyinterpose](#) function immediately before it calls the [Body](#) function.

For more information, see [Specifying Interpose Function Parameters](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.19

<	>
---	---

Common Parameters : [Interpose Functions Parameters](#) : PrePatF

PrePatF

The interpose function, plus any arguments (PrePatFArgs), that is called before each pattern burst. PrePatF is called whenever a pattern burst is started, regardless of how: for example, a VBT start, startstop, or restart. PostPatF is called the first time a burst is seen to have halted, regardless of how it was started.

The PrePatF and PostPatF functions are called before or after each pattern burst. If you specify a list of patterns or pattern sets, the interpose functions are executed multiple times, one before and one after each pattern or pattern set in the list.

For more information, see [Specifying Interpose Function Parameters](#).

<	>
---	---

tlCommonParams.9.20

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : PreTestF

PreTestF

The interpose function, plus any arguments (PreTestFArgs), that is called before the test is executed or the measurement is made.

If there is a list of pins, PreTestF and PostTestF execution varies with the different parametric VBT tests. For some, the functions are executed once for the entire list, before the test on the first pin and after the test on the last pin. For others, the functions are executed before and after each pin in the list. See the VBT test code for each test type to determine how it calls these interpose functions. For more information, see [Specifying Interpose Function Parameters](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.21

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : PostTestF

PostTestF

The interpose function, plus any arguments (PostTestFArgs), that is called after the test is executed or the measurement is made.

If there is a list of pins, PreTestF and PostTestF execution varies with the different parametric VBT tests. For some, the functions are executed once for the entire list, before the test on the first pin and after the test on the last pin. For others, the functions are executed before and after each pin in the list. See the VBT test code for each test type to determine how it calls these interpose functions. For more information, see [Specifying Interpose Function Parameters](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.22

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : PostPatF

PostPatF

The interpose function, plus any arguments (PostPatFArgs), that is called after each pattern burst. PrePatF is called whenever a pattern burst is started, regardless of how: for example, a VBT start, startstop, or restart. PostPatF is called the first time a burst is seen to have halted, regardless of how it was started.

The PrePatF and PostPatF functions are called before or after each pattern burst. If you specify a list of patterns or pattern sets, the interpose functions are executed multiple times, one before and one after each pattern or pattern set in the list.

The PostPatF function is executed after all sites have halted, even if pattern threading is turned on. See [Pattern Threading](#).

For more information, see [Specifying Interpose Function Parameters](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.23

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : EndOfBodyF

EndOfBodyF

The interpose function, plus any arguments (EndOfBodyFArgs), that is called as the last operation in the VBT test body.

In a VBT test, the main test function calls the EndOfBodyinterpose function immediately after it calls the Body function.

For more information, see [Specifying Interpose Function Parameters](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.24

<	>
-----------------	-----------------

Common Parameters : [Interpose Functions Parameters](#) : MeasF

MeasF

The interpose function, plus any arguments (MeasFArgs), that is called before a serial measurement is made.

Only the [DCVIPowerSupply_T](#) test uses this interpose function, and only if the sites are measured serially. For the conditions under which sites are measured serially, see [SerializeMeas](#) in the description of the [DCVIPowerSupply_T](#) test.

If sites are being measured serially, this interpose function is called before each site measurement. For more information, see [Specifying Interpose Function Parameters](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.25

<	>
---	---

Common Parameters : [Interpose Functions Parameters](#) : Specifying Interpose Function Parameters

Specifying Interpose Function Parameters

Each interpose function has two parameters: the name of the function and the list of arguments that the function takes.

You can select the function name from a drop-down list. The list contains the names of all functions in the workbook that conform to the calling convention for interpose functions. On the calling convention, see [Calling Convention](#). The function must already exist before you specify it.

If the function requires parameters, enter them as a comma-separated list in the Args field for the function. Each comma-delimited item is treated as a string. Do not enquote them.

See also [Validation of Interpose Functions](#).

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.26

<	>
-----------------	-----------------

Common Parameters : Pin States Parameters

Pin States Parameters

These parameters specify various pin states:

- A pin’s start state and init state can be set to Lo, Hi, or Z (see [DriveLoPins](#), [DriveHiPins](#), [DriveZPins](#))
- A pin can be set to “float” (see [FloatPins](#))
- Utility pins can be set to 1 or 0 (see [Util0Pins](#), [Util1Pins](#))
- Pins can be disabled (see [DisablePins](#))

See also [Validation of Pin State Values](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.27

	
---	---

[Common Parameters](#) : [Pin States Parameters](#) : DriveLoPins, DriveHiPins, DriveZPins

[DriveLoPins](#), [DriveHiPins](#), [DriveZPins](#)

The pins whose start and init states are to be set to Lo, Hi, or Z.

[Valid Values](#)

To select pins, click the ellipsis button (...) to bring up the Pin Selection tool, which lets you select pins for each parameter. See [Pins Selection](#). Instead of using the tool, you can also type in a comma-separated list of pin or pin group names.

If a pin appears in more than one of these lists (for example, as a pin in one list and as a member of a pin group in another), it is left in the last state applied. The order of application is Lo, Hi, and Z.

See also [Validation of Pin State Values](#).

[Usage](#)

Entering a pin name for one of these parameters sets both the start state and the init state for the pin.

These arguments provide a convenient way to bias pins to VDriveHi or VDriveLo during DC parametric testing.

[Start State](#)

The start state programs the driver value to be set on the pin as each pattern burst starts. It is applied by the driver before the first D0 edge in the pattern burst.

The Postbody resets the start state to Z.

The default is to have the start state selected automatically, depending on the Format of the first vector of each pattern burst.

Note that, for DriveLoPins and DriveHiPins, this setting affects the drive state of the channels just before the pattern start.

Note that the start state applies to all pattern starts in a pattern list or set. So, if you program a start state and run a pattern set in non-burst mode, then IG-XL applies the start state before each pattern in the set. If you need to preserve the pin state between subsequent patterns in the

sequence, then you must include a post-pat interpose function that sets the start state to none (use TheHdw.Digital.Pins(PinList).StartState = chStartNone).

Init State

Setting the init state causes the pin to drive the specified value. The init state is set once, during the Prebody, before the first pattern burst. The Postbody does not reset the init state.

The default is to leave the pin driving whatever value it last drove during the previous pattern burst.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlCommonParams.9.28

<	>
-----------------	-----------------

Common Parameters : [Pin States Parameters](#) : FloatPins

FloatPins

Pins to "float" by opening the relay connections.

Valid Values

To select pins, click the ellipsis button (...) to bring up the Pin Selection tool, which lets you select pins for each parameter. See [Pins Selection](#). Instead of using the tool, you can also type in a comma-separated list of pin or pin group names.

See also [Validation of Pin State Values](#).

Usage

"Float" means that all relay connections are set open for each specified pin. After testing is complete, any pins that were floated are reconnected with the pin electronics, as one of the final steps in the Body execution.

[Pins specified in a Pinlist](#) argument, if the test instance has one, cannot appear in the FloatPins argument. This also means that a pin specified in one argument cannot be part of a pin group specified in the other argument.

[The order of the pins in the list determines the order in which the pins are floated.](#)

<	>
-----------------	-----------------

tlCommonParams.9.29

<	>
-----------------	-----------------

Common Parameters : Pin States Parameters : Util0Pins, Util1Pins

Util0Pins, Util1Pins

Utility pins that are to be set on (driven to 1) or off (driven to 0) during Prebody execution.

Valid Values

To select pins, click the ellipsis button (...) to bring up the Pin Selection tool, which lets you select pins for each parameter. See Pins Selection. Instead of using the tool, you can also type in a comma-separated list of pin or pin group names.

See also Validation of Pin State Values.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.30

<	>
-----------------	-----------------

[Common Parameters](#) : [Pin States Parameters](#) : DisablePins

DisablePins

Pins that are to be disabled during Prebody execution.

Valid Values

To select pins, click the ellipsis button (...) to bring up the Pin Selection tool, which lets you select pins for each parameter. See [Pins Selection](#). Instead of using the tool, you can also type in a comma-separated list of pin or pin group names. See also [Validation of Pin State Values](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.31

<

>

Common Parameters : Precondition Pattern Parameters

Precondition Pattern Parameters

A precondition pattern is a pattern or pattern set that is run for DUT functional preconditioning. The pattern is executed before the parametric measurement is made.

Preconditioning Pattern Parameters Quick Reference

The following parameters are used to control the preconditioning pattern:

PreconditionPat	The pattern or pattern set to be run.
PcpStartLabel	The label at which execution is to start. By default, execution starts at the beginning of the pattern.
PcpStopLabel	The label at which execution is to stop.
PcpCheckPatGen	Whether to check the pattern generator for pass/fail status and to pass that status to the flow controller:
PcpDisableAlarmCheck	Whether to disable instrument alarms that occur during execution of the preconditioning pattern.

Note that concurrent test is disabled when executing a preconditioning pattern in a VBT test instance.

For more information, see Validation of Patterns and Labels.

<

>

--	--	--

tlCommonParams.9.32

<	>
-----------------	-----------------

Common Parameters : [Precondition Pattern Parameters](#) : PreconditionPat

PreconditionPat

The pattern or pattern set to be run.

Valid Values

To select a pattern, click the ellipsis button (...) to bring up the Pattern Selector tool, which lets you select a pattern file, group, or set. You can also type in a pattern file pathname or pattern set name. For more information, see:

- [Pattern Selector](#)
- [Validation of Patterns and Labels](#)

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.33

<	>
-----------------	-----------------

Common Parameters : [Precondition Pattern Parameters](#) : PcpStartLabel

PcpStartLabel

The label at which execution is to start. By default, execution starts at the beginning of the pattern.

Valid Values

Enter the name of a label. The label must be declared in the pattern file as a start label, global label, or global subr label.

See also [Validation of Patterns and Labels](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.34

<
>

Common Parameters : Precondition Pattern Parameters : PcpStopLabel

PcpStopLabel

The label at which execution is to stop.

Valid Values

Enter the name of a label. The label must be declared in the pattern file as a start label, global label, or global subr label.

If a stop label is specified, pattern execution runs to the stop label or until a time-out occurs.

See also [Validation of Patterns and Labels](#).

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.35

<	>
-----------------	-----------------

Common Parameters : [Precondition Pattern Parameters](#) : PcpCheckPatGen

PcpCheckPatGen

Whether to check the pattern generator for pass/fail status and to pass that status to the flow controller.

Valid Values

1	Enable. The status is checked and is returned to the flow controller. If the PreconditionPat pattern fails, the test fails.
0	Disable. The pattern generator is not checked. Even if the PreconditionPat pattern fails, this failure is not reflected in the test result that is returned to the flow controller.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.36

<	>
-----------------	-----------------

Common Parameters : [Precondition Pattern Parameters](#) : PcpDisableAlarmCheck

PcpDisableAlarmCheck

Whether to disable instrument alarms that occur during execution of the preconditioning pattern. Note that not all VBT test types that support a preconditioning pattern include this parameter.

Valid Values

True	Disable the alarm window from being opened for the pattern burst.
False	Do not disable the alarm window.

Usage

By default, if a test bursts a pattern, the alarm window is opened and remains open until the pattern stops. This can cause a problem with unwanted alarms. For example, if the pattern is simply driving the clock pin, changes to the instrument will cause transient or temporary alarms. Because the alarm window is open, alarms are returned when the instrument measurement is made. To avoid reporting these alarms, set this parameter to True. This disables the alarm window from being opened for the preconditioning pattern burst.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlCommonParams.9.37

<	>
-----------------	-----------------

Common Parameters : Timing and Levels Tab

Timing and Levels Tab

The Timing and Levels tab, which is common to all test types, specifies the context for executing the test. The context includes:

- | | |
|---|--|
| • | TimeSet, EdgeSet, Pin Levels |
| • | Mixed-Sig |
| • | Overlay |
| • | AC Selector, AC Category, DC Selector, DC Category |
| • | Serial Timing |
| • | Jitter |
| • | Time Domains |

The fields of the tab let you select from sheets and other items that are defined in the IG-XL workbook. Each field is a drop-down list of the relevant items in the workbook. You select one of the items from the list.

Specify these values only if the test instance needs them. Otherwise, leave them blank. The default for all these fields is a blank.

TimeSet, EdgeSet, Pin Levels

The sheets from the test program workbook that are to be used to define settings for timing (TimeSet, EdgeSet) and pin levels (PinLevels) for this test instance.

The drop-down list for each field contains the names of all of the corresponding sheets in the workbook. The TimeSet list contains all Time Sets and Time Sets (Basic) sheets.

If TimeSet identifies a Time Sets (Basic) sheet, rather than a Time Sets sheet, then do not specify an EdgeSet.

If a test instance does not use timing or pin level values, then you need not specify a sheet of that type. For example, a test instance that does not run patterns does not need time sets or edge sets.

Mixed-Sig

The mixed-signal timing set that is to be used for this test instance.

The drop-down list contains the names of all sets defined on the currently active Mixed-Signal Timing sheet.

Overlay

The name of the Adjust setup whose adjusted spec value is to be overlaid on the spec variable of the same name in this test instance.

The drop-down list contains the names the Adjust setups that are defined on the currently active Characterization sheet. The menu displays the names of all setups that contain at least one Adjust row.

If the specified setup has not yet been executed, there is no overlay value, and the non-adjusted hardware value is used.

Note that the adjusted value generated by an Adjust setup remains available from one device to the next, as long as the program is running, unless the value is overwritten by a subsequent execution of the same setup.

For more information, see Performing Adjust and Measure in the DataTool section

AC Selector, AC Category, DC Selector, DC Category

For each spec symbol on an AC or DC Specs sheet, the combination of category and selector defines the current value of the symbol. You specify the selector and category to be used for AC spec symbols and for DC spec symbols.

The drop-down list contain the selectors and categories defined on the AC and DC Specs sheets that are specified by the current job. (AC and DC Specs sheets are specific to a job. Selectors and categories defined on the specs sheets are specific to a test instance in the job.)

If a test instance does not use AC or DC spec symbols, then you need not specify a selector and category of that type. If you do specify either a selector or category, however, you must specify the other item of that type. For example, if you specify a DC category, you must also specify a DC selector.

Serial Timing

The Serial Timing sheet name to add to the Time Sets column of the Test Instances sheet that specifies SB6G or UltraSerial10G timing separate from the digital timing. For more information, see

Serial Timing Sheet.

Jitter

The Jitter sheet name to add to the Time Sets column of the Test Instances sheet that specifies the details of the jitter insertion or measurement. For more information, see [Jitter Sheet](#).

Time Domains

The Time Domains area allows you to set up or change the time domains used in the VBT test. You can set up multiple time domains using this interface, where each tab represents a different time domain setup.

Time Domains	This checkbox enables and disables the time domains.
Time Domain	This pull down menu selects the time domain name.
Add	This button includes a new tab for that time domain name.
Remove	This button removes the selected time domain tab.
Time Set	This pull down menu lists valid time sets for the selected time domain.
Edge Set	This pull down menu lists valid edge sets for the selected time domain.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlCommonParams.9.38

<	>
-----------------	-----------------

Common Parameters : Limits Tab

Limits Tab

The Limits tab provides a convenient way to supply values for a limits check: the high and low limits, the test number and name for the limits check, and the passing and failing bin. The tab contains a grid, with a row for each limits check in the test.

This tab works with the Test Limits in Flow feature. If a VBT test uses this feature, the limits and other related values are specified in the Flow Table sheet, instead of in the TestLimit method in the test itself. The values for each limits check in a test is stored in a row with a Use-Limit row that follows the Test row for the test. Because a VBT test can perform several limits checks, there must be one Use-Limit row for each limits check in the test.

The Limits tab lets you edit the values for limits checks from the instance editor, as well as directly on the Flow Table sheet. Any values you enter or edit on the Limits tab are written directly to the Flow Table.

The Limits tab contains the limits check grid only if you invoke the instance editor from the Flow Table sheet: for example, by right-clicking the test instance name in the Command Parameter column and selected Edit>Instance or Definition>Instance.

If you invoke the instance editor from the Test Instances sheet, the Limits tab is blank. The reason that the tab is blank is that a single test instance can be used several times in a Flow Table, possibly with different test limits specified for each use. If you invoke the instance editor from the Test Instances sheet, IG-XL cannot know which limits values are to be used, so the Limits tab is blank. If you invoke the instance editor from the Flow Table, the test instance is bound to the limits values in the rows where you right-clicked, so IG-XL can display those limits values on the Limits tab.

For more information, see Test Limits in Flow.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcviPowerSupply.3.01

<	>
-----------------	-----------------

DCVI Power Supply VBT Test

DCVI Power Supply VBT Test

The DCVI Power Supply VBT test programs various elements of the DC30, DC75, and UltraVI80 DCVI instruments. This VBT test type allows you to do the following:

- | | |
|--|---|
| | <ul style="list-style-type: none">Assign the power pins to test |
| | <ul style="list-style-type: none">Set up multiple pattern control tests |
| | <ul style="list-style-type: none">Select a main or alternate voltage |
| | <ul style="list-style-type: none">Set main and alternate voltage values |
| | <ul style="list-style-type: none">Select a current range |
| | <ul style="list-style-type: none">Set the current clamp level |
| | <ul style="list-style-type: none">Select the gate state before connecting the power supply |
| | <ul style="list-style-type: none">Select how to perform site measurements (serially or in parallel) |
| | <ul style="list-style-type: none">Set the number of samples to take |
| | <ul style="list-style-type: none">Set a sampling time |

- [Set the settling time](#)

- [Set high and low power limits \(see Limits Tab\)](#)

For a list of parameters used in this VBT test, see [DCVI Power Supply VBT Test Parameters](#).
For information on other parameter groups used in this VBT test, see:

- [Hold State Pattern Parameters](#)

- [Interpose Functions Parameters](#)

- [Pin States Parameters](#)

- [Precondition Pattern Parameters](#)

- [Timing and Levels Tab](#)

- [Limits Tab](#)

For more information, see:

- [About the DCVI](#)

- [DCVI Theory of Operation](#)

- [DCVI Programming](#)

- [About the UltraVI80 DCVI](#)

- [UltraVI80 DCVI Theory of Operation](#)

- [UltraVI80 DCVI Programming](#)

- [DCVI \(VBT syntax\)](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

tlDcviPowerSupply.3.02

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters

DCVI Power Supply VBT Test Parameters
DCVI Power Supply Test parameters are summarized below.

PowerPins	Specifies the DUT power supply pins or pin groups to be tested.
TestControl	Selects a single or multiple device conditioning setup.
VoltageOutput	Selects which voltage output to use during test: main, alternate, or ambient (inherit the previous voltage output setup).
MainVoltage	Sets the main force voltage value for the DCVI instrument.
AltVoltage	Sets the alternate force voltage value for the DCVI instrument.
IRange	Selects the current range of the DCVI during measurement.
CurrentClamp	Sets DCVI clamping voltage during measurement.
RelayMode	Selects the DCVI operational switching mode before connecting the power supply: unpowered (default) or powered (hot).
SerializeMeas	Whether to measure sites serially instead of in parallel when sharing power pins.
SampleSize	Sets the number of samples to take and average during test measurement.
SamplingTime	Sets the total time in seconds during which the DCVI is to take measurement samples.
SettlingTime	Sets the wait time in seconds before the DCVI takes the measurement.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcviPowerSupply.3.03

<	>
---	---

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : PowerPins

PowerPins

Specifies the DUT power supply pins or pin groups to be tested.

Valid Values

Specify valid power supply pins or pin groups. The menu contains any pin on the Pin Map sheet whose type is Power, plus any pin group containing only pins whose type is Power. No default value.

Usage

You must specify a pin list by either entering a comma-separated list of pin or pin group names in the Value data box or selecting pins and pin groups from the Pins Selection data box. Click the ellipsis button (...) to bring up the Pins Selection data box to select pins and pin groups. The Available Pins data box contains the names of all power supply pins and pin groups on the Pin Map sheet. The order of the pins in the list determines the order in which the pins will be tested.

Related Topics

- Pins Selection

<	>
---	---

tlDcviPowerSupply.3.04

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : TestControl

TestControl
Selects a single or multiple device conditioning setup.
Valid Values
Values are available from a drop-down list:

psSingleTest	Makes a pass/fail determination based on a single device conditioning setup. Default.
psPatFlagMultipleTests	Makes multiple device conditioning setups possible within a single hold-state pattern.

Usage
Pattern Flag Multiple Tests
This option makes possible multiple device conditioning setups within a single hold-state pattern. For each test you must enter a setup, which consists of the HiLimit and LoLimit arguments and the SettlingTime argument. To create additional tests you must add an additional Use-Limit in the Flow Sheet for each test as shown:

Command		Limits			
Opcode	Parameter	TName	TNum	LoLim	HiLim
Test	MyTest	current			
Use-Limit	MyTest			1	2
Use-Limit	MyTest			2	3
Use-Limit	MyTest			3	4

Multiple Tests in the Flow Sheet
The number of settling time entries must match the number of Use-Limit in the Flow Table. Each settling time must be separated by a # symbol. Having three tests would require an entry similar to the following in the SettlingTime menu:
0.03#0.01#0.02

During execution of a hold-state pattern, when the flag conditions are first met, the power supply makes its test using the values in the first setup and makes a pass/fail determination. The flags are then reversed, and pattern execution continues until the flag conditions are met a second time. At that point, the power supply makes its test using the values in the second setup. The process continues in this way.

This option is useful for IDDQ testing, which requires multiple static modes on the DUT.

Related Topics

- [SettlingTime](#)
- [Hold State Pattern Parameters](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlDcviPowerSupply.3.05

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : VoltageOutput

VoltageOutput
Selects which voltage output to use during test.
Valid Values
Values are available from a drop-down list:

psSourceMain	Use the main voltage value. Default.
psSourceAlternate	Use the alternate voltage value.
psSourceAmbient	Use whatever voltage value is currently in the hardware. This value is inherited the previous voltage output setup and may have been programmed by the most recently loaded Pin Levels sheet (either by this test instance or a previous test instance) or by an interpose function.

Related Topics

- MainVoltage
- AltVoltage

<

>

--	--	--

tlDcviPowerSupply.3.06

<	>
---	---

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : MainVoltage

MainVoltage

Sets the main force voltage value for the DCVI instrument.

Valid Values

Enter a voltage value in decimal from -1000 to +1000 V, a single spec variable, or an equation. No default value.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [Expression Builder](#)
- [VoltageOutput](#)
- [AltVoltage](#)

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcviPowerSupply.3.07

<	>
-----------------	-----------------

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : AltVoltage

AltVoltage

Sets the alternate force value for the DCVI instrument. The ability to specify an alternate voltage can be useful during debugging.

Valid Values

Enter voltage values in decimal from -1000 to +1000 V, a single spec variable, or an equation. No default value.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|---|------------------------------------|
| • | Expression Builder |
| • | VoltageOutput |
| • | MainVoltage |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcviPowerSupply.3.08

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : IRange

IRange
Selects the current range of the DCVI during measurement.
Valid Values
Values are available from a drop-down list:

- 2mA
- 20mA
- 200mA (default)
- 400mA
- 1A
- 2A
- 4A
- Auto (Use the high and low limits to determine the best range to use.)

☒ **Note:**
This DCVI-specific argument is not the same as the DCVS MeterCurrentRange argument, which determines the appropriate hardware meter current range setting based on the value entered.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tlDcviPowerSupply.3.09

<	>
-----------------	-----------------

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : CurrentClamp

CurrentClamp

Sets DCVI clamping voltage during measurement. This is the value at which the instrument limits current or voltage. If voltage is being forced, this value specifies the current clamp. If current is being forced, this value specifies the voltage clamp.

Valid Values

Enter a current clamp value, a single spec variable, or an equation. The VBT test uses the maximum range value as the current clamp level.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [Expression Builder](#)

<	>
-----------------	-----------------

tlDcviPowerSupply.3.10

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : RelayMode

RelayMode
Selects the DCVI operational switching mode before connecting the power supply.
Valid Values
Values are available from a drop-down list:

tlUnpowered	Unpowered relay signal path (cold switching). Default,
tlPowered	Powered relay signal path (hot switching).

Usage
By default, an instrument’s forcing function and DUT power are turned off when relay switching tasks are performed. You can cause the forcing function and DUT power to remain on by setting this argument to tlPowered, thereby enabling hot switching.
Using powered relay mode can result in a moderate performance improvement. Note, however, that hot switching can result in premature failure of relays.
When applying levels and timing to the DCVI, the test instance follows these connection and gating sequences for the unpowered and powered relays modes:
Unpowered Relay Mode Sequence

1.

Gate off single pin.
2.

Set current range to Auto.
3.

Set voltage range to Auto.

4. [Set voltage to 0.0 V.](#)
5. [Wait 5 ms.](#)
6. [Connect single pin.](#)
7. [Go to step 1 and repeat for each pin.](#)
8. [Write level values for a single pin.](#)
9. [Gate on that pin.](#)
10. [Hard wait for the Tdelay.](#)
11. [Go to step 8 and repeat for each pin.](#)

Powered Relay Mode Sequence

1. [Check pin connection state:](#)
 - a. [If pin is not connected, go to step 2.](#)
 - b. [If pin is connected, go to step 5](#)
2. [Gate off single pin.](#)
3. [Wait 5 ms.](#)
4. [Connect single pin.](#)
5. [Go to step 1 and repeat for each pin until all pins are connected.](#)
6. [Write level values for a single pin.](#)

- | | |
|----|--|
| 7. | Gate on that pin. |
| 8. | Hard wait for the Tdelay. |
| 9. | Go back to step 6 and repeat for each pin. |

	
---	---

tlDcviPowerSupply.3.11

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : SerializeMeas

SerializeMeas

Whether to measure sites serially instead of in parallel when sharing power pins.

Valid Values

Values are available from a drop-down list:

False	Do not measure sites serially. Default.
True	Measure sites serially when sharing power pins. See Usage below.

Usage

Note the following behavior when using this parameter:

- If the channel map has no shared power pins, then the state of this control has no effect on testing and all sites are measured in parallel.
- If the channel map includes shared power pins and this control is not selected, then all sites are measured in parallel.
- If the channel map includes shared power pins and the control is selected, then the sites are measured one at a time in a loop. Performance may be degraded as each site must be measured one at a time, instead of all being measured at the same time.

If sites are measured serially, IG-XL executes the MeasF interpose function before each site measurement is made.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcviPowerSupply.3.12

<	>
-----------------	-----------------

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : SampleSize

SampleSize

Sets the number of samples to take and average during test measurement. These samples (that is, the number of analog-to-digital conversions) are used to obtain the test result.

Valid Values

Enter a sample size value, a single spec variable, or an equation. The maximum sample size value is 10000. No default.

Usage

If neither SampleSize nor SamplingTime is specified, the VBT test uses a sample count of 1.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|---|------------------------------------|
| • | Expression Builder |
|---|------------------------------------|
- | | |
|---|------------------------------|
| • | SamplingTime |
|---|------------------------------|
- | | |
|---|------------------------------|
| • | SettlingTime |
|---|------------------------------|

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcviPowerSupply.3.13

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : SamplingTime

SamplingTime

Sets the total time in seconds during which the DCVI is to take measurement samples.

Valid Values

Enter a sampling time value in seconds, a single spec variable, or an equation.

Usage

If this value is specified, the number of samples equals the SamplingTime value divided by the test system conversion time. In this way, the sampling can be independent of conversion speed.

SamplingTime, if specified, preempts any SampleSize value.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation into the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [Expression Builder](#)
- [SampleSize](#)
- [SettlingTime](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcviPowerSupply.3.14

<

>

DCVI Power Supply VBT Test : DCVI Power Supply VBT Test Parameters : SettlingTime

SettlingTime

Sets the wait time in seconds before the DCVI takes the measurement. This pause before measurement allows the force value to settle and stabilize. It is measured from the time the DCVI is gated on to the time the measurement is taken, that is, to the start of the analog-to-digital conversion.

Valid Values

Enter a settling time value in seconds, a single spec variable, or an equation. The maximum settling time is 30 seconds. Default is 0 seconds.

Usage

If you are doing multiple power supply tests (for instance, IDDQ), you must set a settling time value for each test, with each settling time value separated by a # symbol. You cannot use the Expression Builder. The number of settling time values must equal the number of test limits being applied. The time required before measurement is usually an attribute of the test system hardware, not of the DUT. However, DIBs with unusually high wiring capacitance, or with lump capacitance, may require a higher-than-normal settling time.

Related Topics

- TestControl
- SampleSize
- SamplingTime

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcvsPowerSupply.4.01

<	>
-----------------	-----------------

DCVS Power Supply VBT Test

DCVS Power Supply VBT Test

The DCVS Power Supply VBT Test programs various elements of the DCVS instrument on the HexVS, HDVS, and UVS256, and VSM instrument boards. This VBT test type allows you to do the following:

- Assign the power pins to test
- Set up multiple pattern control tests
- Select a main or alternate voltage
- Set main and alternate voltage values
- Select a meter current range
- Select the gate state before connecting the power supply
- Set the number of samples to take
- Set a sampling time
- Set the settling time

- [Set the current fold limit level](#)

- [Set high and low power limits \(see Limits Tab\)](#)

For a list of parameters used in this VBT test, see [DCVS Power Supply VBT Test Parameters](#).
For information on other parameter groups used in this VBT test, see:

- [Hold State Pattern Parameters](#)

- [Interpose Functions Parameters](#)

- [Pin States Parameters](#)

- [Precondition Pattern Parameters](#)

- [Timing and Levels Tab](#)

- [Limits Tab](#)

For more information, see:

- [About the HexVS](#)

- [About the HDVS](#)

- [About the UltraVS256](#)

- [About the VSM](#)

- [DCVS \(VBT Syntax\)](#)



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcvsPowerSupply.4.02

<

>

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters

DCVS Power Supply VBT Test Parameters
The DCVS Power Supply Test parameters are summarized below.

PowerPins	Specifies the DUT power supply pins or pin groups to be tested.
TestControl	Selects a single or multiple device conditioning setup.
VoltageOutput	Selects which voltage output DAC to use during test: main, alternate, or ambient (inherit previous voltage output setup).
MainVoltage	Sets the main voltage DAC value used for the test.
AltVoltage	Sets the alternate voltage DAC value used for the test.
CurrentRanges	Selects the desired meter and output current measurement ranges. IG-XL chooses the nearest available range
CurrentLimit	Sets the current fold limit level (also known as current clamp level).
RelayMode	Selects the operational gate state before connecting the power supply: unpowered (default) or powered (hot).
SampleSize	Sets the number of samples to take and average during test measurement.
SamplingTime	Sets the total time in seconds during which the DCVS is to take measurement samples.
SettlingTime	Sets the wait time in seconds before the DCVS takes the measurement.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcvsPowerSupply.4.03

<	>
---	---

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : PowerPins

PowerPins

Specifies the DUT power supply pins or pin groups to be tested.

Valid Values

Specify valid power supply pins or pin groups. The menu contains any pin on the Pin Map sheet whose type is Power, plus any pin group containing only pins whose type is Power. No default value.

Usage

You must specify a pin list by either entering a comma-separated list of pins or pin group names in the Value data box or selecting pins and pin groups from the Pins Selection data box. Click the ellipsis button (...) to bring up the Pins Selection data box to select pins and pin groups. The Available Pins data box contains the names of all power supply pins and pin groups on the Pin Map sheet. The order of the pins in the list determines the order in which the pins will be tested.

Related Topics

- Pins Selection

<	>
---	---

tlDcvsPowerSupply.4.04

<

>

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : TestControl

TestControl

Selects a single or multiple device conditioning setup.

Valid Values

Values are available from a drop-down list:

psSingleTest	Makes a pass/fail determination based on a single device conditioning setup. Default.
psPatFlagMultipleTests	Makes multiple device conditioning setups possible within a single hold-state pattern.

Usage

Pattern Flag Multiple Tests

This option makes possible multiple device conditioning setups within a single hold-state pattern. For each test you must enter a setup, which consists of the HiLimit and LoLimit arguments and the SettlingTime argument. To create additional tests you must add an additional Use-Limit in the Flow Sheet for each test as shown:

Command		Limits			
Opcode	Parameter	TName	TNum	LoLim	HiLim
Test	MyTest	current			
Use-Limit	MyTest			1	2
Use-Limit	MyTest			2	3
Use-Limit	MyTest			3	4

Multiple Tests in the Flow Sheet

The number of settling time entries must match the number of Use-Limit in the Flow Table. Each settling time must be separated by a # symbol. Having three tests would require an entry similar to the following in the SettlingTime menu:

0.03#0.01#0.02

During execution of a hold-state pattern, when the flag conditions are first met, the power supply makes its test using the values in the first setup and makes a pass/fail determination. The flags are then reversed, and pattern execution continues until the flag conditions are met a second time. At that point, the power supply makes its test using the values in the second setup. The process continues in this way.

This option is useful for IDDQ testing, which requires multiple static modes on the DUT.

Related Topics

- [SettlingTime](#)
- [Hold State Pattern Parameters](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlDcvsPowerSupply.4.05

<	>
-----------------	-----------------

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : VoltageOutput

VoltageOutput
Selects which voltage output DAC to be used for the test.
Valid Values
Values are available from a drop-down list:

psSourceMain	Use the main voltage value. Default.
psSourceAlternate	Use the alternate voltage value.
psSourceAmbient	Use whatever voltage value is currently in the hardware. This value is inherited the previous voltage output setup and may have been programmed by the most recently loaded Pin Levels sheet (either by this test instance or a previous test instance) or by an interpose function.

Related Topics

- | | |
|---|-------------|
| • | MainVoltage |
|---|-------------|
- | | |
|---|------------|
| • | AltVoltage |
|---|------------|

<	>
-----------------	-----------------

--	--	--

tlDcvsPowerSupply.4.06

<	>
-----------------	-----------------

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : MainVoltage

MainVoltage

Sets the main voltage value used for the test.

Valid Values

Enter a voltage value in decimal, a single spec variable, or an equation. No default value.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|---|------------------------------------|
| • | Expression Builder |
| • | VoltageOutput |
| • | AltVoltage |

<	>
-----------------	-----------------

--	--	--

tlDcvsPowerSupply.4.07

<	>
---	---

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : AltVoltage

AltVoltage

Sets the alternate voltage value used for the test. The ability to specify an alternate voltage can be useful during debugging.

Valid Values

Enter a voltage value in decimal from, a single spec variable, or an equation. No default value.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [Expression Builder](#)
- [VoltageOutput](#)
- [MainVoltage](#)

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcvsPowerSupply.4.08

<

>

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : CurrentRanges

CurrentRanges

Selects the desired meter and output current ranges. Based on the value entered, IG-XL determines the appropriate hardware meter and output current range setting.

Valid Values

Values are available from a drop-down list:

HDVS	=10*uA, =20*uA, =40*uA, =100*uA, =200*uA, =400*uA, =1*mA, =2*mA, =4*mA, =10*mA, =20*mA, =40*mA, =100*mA, =200*mA, =400*mA, =1*A, =2*A, =4*A
HexVS	=10*mA, =100*mA, =1*A, =15*A, =30*A, =60*A, =90*A
UVS256	=4*uA, =10*uA, =20*uA, =40*uA, =100*uA, =200*uA, =400*uA, =1*mA, =2*mA, =4*mA, =10*mA, =20*mA, =40*mA, =100*mA, =200*mA, =400*mA, =500*mA, =800*mA, =1*A, =2*A, =4*A
VSM	=1*A, =2*A, =3*A, =4*A, =5*A, =6*A, =7*A, =8*A, =9*A, =10*A, =11*A, =21*A, =51*A, =81*A, =88*A, =99*A, =102*A, =105*A, =110*A, =121*A, =126*A, =132*A, =143*A, =147*A, =153*A, =154*A, =162*A, =165*A, =168*A, =176*A, =189*A, =204*A, =210*A, =231*A, =243*A, =252*A, =255*A, =273*A, =294*A, =306*A, =315*A, =324*A, =336*A, =357*A, =405*A, =408*A, =459*A, =486*A, =510*A, =561*A, =567*A, =612*A, =648*A, =663*A, =714*A, =729*A, =765*A, =810*A, =816*A, =891*A, =972*A, =1053*A, =1134*A, =1215*A, =1296*A

No default value.

Usage

The value specified is used to specify both the output current and meter current ranges. IG-XL automatically fits the requested value to the closest available range.

Related Topics

For more information, see [Current Meter Setup](#), [Strobe](#), and [Readback Programming](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcvsPowerSupply.4.09

<	>
---	---

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : CurrentLimit

CurrentLimit

Sets the current fold limit level (also known as current clamp level). This is the value at which the instrument limits current.

Valid Values

Enter a current limit value, a single spec variable, or equation. The VBT test uses the maximum range value as the current fold limit level.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|---|--|
| • | Expression Builder |
| • | Setting the Current Limit. |

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlDcvsPowerSupply.4.10

<	>
-----------------	-----------------

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : RelayMode

RelayMode
Selects the operational gate state before connecting the power supply.
Valid Values
Values are available from a drop-down list:

tlUnpowered	Unpowered relay signal path (cold switching). Default.
tlPowered	Powered relay signal path (hot switching).

Usage
By default, an instrument’s forcing function and DUT power are turned off when relay switching tasks are performed. You can cause the forcing function and DUT power to remain on by setting this argument to tlPowered, thereby enabling “hot” switching.
Using powered relay mode can result in a moderate performance improvement. Note, however, that hot switching can result in premature failure of relays.
For the Functional VBT test, the default is tlPowered (hot switching).
For all other VBT tests, the default is tlUnpowered (cold switching).

<	>
-----------------	-----------------

tlDcvsPowerSupply.4.11

<	>
-----------------	-----------------

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : SampleSize

SampleSize

Sets the number of samples to take and average during test measurement. These samples (that is, the number of analog-to-digital conversions) are used to obtain the test result.

Valid Values

Enter a sample size value, a single spec variable, or an equation. The maximum sample size value is 10000. No default.

Usage

If neither SampleSize nor SamplingTime is specified, the VBT test uses a sample count of 1. To enter an equation, click the ellipsis button to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area. Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|--|--|
| | <ul style="list-style-type: none">Expression Builder |
| | <ul style="list-style-type: none">SamplingTime |
| | <ul style="list-style-type: none">SettlingTime |

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlDcvsPowerSupply.4.12

<	>
-----------------	-----------------

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : SamplingTime

SamplingTime

Sets the total time in seconds during which the DCVS is to take measurement samples.

Valid Values

Enter a sampling time value in seconds, a single spec variable, or an equation.

Usage

If this value is specified, the number of samples equals the SamplingTime value divided by the test system conversion time. In this way, the sampling can be independent of conversion speed.

SamplingTime, if specified, preempts any SampleSize value.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|--|--|
| | <ul style="list-style-type: none">Expression Builder |
| | <ul style="list-style-type: none">SampleSize |
| | <ul style="list-style-type: none">SettlingTime |

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlDcvsPowerSupply.4.13

<

>

DCVS Power Supply VBT Test : DCVS Power Supply VBT Test Parameters : SettlingTime

SettlingTime

Sets the wait time in seconds before the DCVS takes the measurement. This pause before measurement allows the force value to settle and stabilize. It is measured from the time the DCVS is gated on to the time the measurement is taken, that is, to the start of the analog-to-digital conversion.

Valid Values

Enter a settling time value in seconds, a single spec variable, or an equation. The maximum settling time is 30 seconds. Default is 0 seconds.

Usage

If you are doing multiple power supply tests (for instance, IDDQ), you must set a settling time value for each test, with each settling time value separated by a # symbol. You cannot use the Expression Builder. The number of settling time values must equal the number of test limits being applied. The time required before measurement is usually an attribute of the test system hardware, not of the DUT. However, DIBs with unusually high wiring capacitance, or with lump capacitance, may require a higher-than-normal settling time.

Related Topics

- [TestControl](#)
- [SampleSize](#)
- [SamplingTime](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlEmpty

<	>
-----------------	-----------------

Empty VBT Test

Empty VBT Test

The Empty VBT test has the optional parameters common to all VBT tests, which are used to execute interpose functions and set levels, timing, and pins. You can use Empty VBT tests to set any of these values without actually executing a test.

For information on common VBT test parameters, see:

- [Interpose Functions Parameters](#)
- [Pin States Parameters](#)
- [Timing and Levels Tab](#)

<	>
-----------------	-----------------

tlFunctional.6.01

<	>
-----------------	-----------------

Functional VBT Test

Functional VBT Test

The Functional VBT test programs various elements of a digital instrument (HSD1000, UltraPin800, or UltraPin1600). This VBT test type allows you to do the following:

- [Set up functional test patterns for execution](#)
- [Enable pattern threading](#)
- [Specify a pattern timeout period](#)
- [Set the operational switching mode](#)
- [Get pass/fail result reports](#)

For a list of parameters used in this VBT test, see [Functional VBT Test Parameters](#).
For information on the other groups of parameters used in this VBT test, see:

- [Flag Match Parameters](#)
- [Interpose Functions Parameters](#)
- [Pin States Parameters](#)

- [Timing and Levels Tab](#)

Note that the Limits tab is not used in this VBT test type.
For more information, see:

- [HSD1000 Programming](#)

- [UltraPin800 Programming](#)

- [UltraPin1600 Programming](#)

- [Digital](#) (VBT Syntax)

<

>

tlFunctional.6.02

<

>

Functional VBT Test : Functional VBT Test Parameters

Functional VBT Test Parameters

Functional VBT test parameters are summarized below. These parameters appear at the top of the instance editor and are not grouped under a label. They appear here in the order in which they appear on the instance editor.

Patterns	The patterns, pattern sets, and pattern groups to execute.
ReportResult	Whether to return the pass/fail result.
PatThreading	Enables pattern threading, which is the capability of executing patterns independently at each site.
PatternTimeout	The timeout period for pattern execution.
RelayMode	Operational switching mode: unpowered (default) or powered (hot).
ResultMode	Specifies how to return pass/fail results for pattern sets and time domains with the burst option set to Yes.
ConcurrentMode	Specifies the concurrent mode to be used when executing a pattern.
WaitTimeDomain	Specifies the time domains that IG-XL will wait for based on the CPU wait flags states specified in the WaitFlagA, WaitFlagB, WaitFlagC, and WaitFlagD, parameters

For information on other groups of parameters used in this VBT test, see Common Parameters.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlFunctional.6.03

<

>

Functional VBT Test : Functional VBT Test Parameters : Patterns

Patterns

The patterns and pattern sets to execute for functional testing. Note that pattern groups are not supported in UltraFLEX.

Valid Values

Specify a valid pattern or a comma-separated list of patterns in the Value field or select patterns from the Pattern Selector window. No default value.

Usage

You must specify at least one pattern to be executed. The order of the patterns in the list determines the order in which they are executed. You can specify either an uncompressed (.pat) or a compressed (.pat.gz) pattern file.

Click the ellipsis button (...) to bring up the Pattern Selector data box to select patterns. You can use the Disk Patterns control to select a pattern on a disk or the Pattern Groups and Sets control to choose pattern sets defined in the workbook.

Concurrent Patterns

To specify patterns that are to run concurrently, use the plus sign (+) to join them. For example, the following specifies that Pat1, Pat2, and Pat3 are to be executed concurrently:

Pat1+Pat2+Pat3

Related Topics

- Pattern Selector
- Pattern Threading and MTD Patterns
- PatThreading

•	PatternTimeout
---	--------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlFunctional.6.04

<

>

Functional VBT Test : Functional VBT Test Parameters : ReportResult

ReportResult

Whether the pass/fail result is to be returned to the flow controller and the datalogger.

Valid Values

Values are available from a drop-down list:

pfAlways	Always returns results to the flow controller and datalogger. Default.
pfNever	Never returns results to the flow controller and datalogger.
pfFailsOnly	Returns only failing tests to the flow controller and datalogger.
pfAlwaysOld	Same as pfAlways. Added for backward compatibility. Do not select for new tests.

Usage

This parameter specifies whether to set the pass/fail status of the test, as well as whether to datalog the result. If you set this to pfNever, a failing test is not returned to the flow controller and therefore cannot cause the device to bin out. The default is pfAlways, which always returns the pass/fail result to the flow controller.

<

>

tlFunctional.6.05

<

>

Functional VBT Test : [Functional VBT Test Parameters](#) : [PatThreading](#)

PatThreading

Enables pattern threading, which is the capability of executing patterns independently at each site.

Valid Values

Values are available from a drop-down list:

False	Disables pattern threading. Multiple sites can share a single pattern generator, with the result that the sites must be executing the same pattern vectors at the same time. Default.
True	Enables pattern threading. Each site's pattern is executed independently, able to make its own branch decisions regardless of the results of execution at other sites.

Related Topics

- [Pattern Threading and MTD Patterns](#)
- [Patterns](#)
- [PatternTimeout](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlFunctional.6.06

<	>
-----------------	-----------------

[Functional VBT Test](#) : [Functional VBT Test Parameters](#) : PatternTimeout

PatternTimeout

The timeout period for pattern execution. This is the maximum time a pattern burst can run before IG-XL issues a halt to stop the pattern burst.

Valid Values

Enter a value in the PatternTimeout box. Time-out is always enabled. The default time-out value is 30 seconds. A value of 0 means to halt execution immediately.

Related Topics

- | | |
|---|--|
| • | Pattern Threading and MTD Patterns |
| • | Patterns |
| • | PatThreading |

<	>
-----------------	-----------------

tlFunctional.6.07

<

>

Functional VBT Test : Functional VBT Test Parameters : RelayMode

RelayMode

Operational switching mode of the instrument. Determines if instruments and power supplies should be powered up during PreBody execution.

Valid Values

Values are available from a drop-down list:

tlUnpowered	Unpowered relay signal path (cold switching). Default.
tlPowered	Powered relay signal path (hot switching).

Usage

By default, an instrument’s forcing function and DUT power are turned off when relay switching tasks are performed. You can cause the forcing function and DUT power to remain on by setting this argument to Powered, thereby enabling hot switching.

Using powered relay mode can result in a moderate performance improvement. Note, however, that hot switching can result in premature failure of relays.

<

>

2015 Teradyne, Inc. All rights reserved.

tlFunctional.6.08

<

>

Functional VBT Test : Functional VBT Test Parameters : ResultMode

ResultMode

Specifies how to return pass/fail results for pattern sets and time domains with the burst option set to Yes.

Valid Values

Values are available from a drop-down list:

tlResultModeOne	Return one pass/fail result for all time domains and pattern modules. If any module on any time domain fails, IG-XL returns a failing result.
tlResultModeDomain	Return a unique pass/fail result for each time domain. Default. (Replaces tlResultModeGanged used in earlier releases.) IG-XL uses a separate test number for each module in each time domain.
tlResultModeModule	Return a unique pass/fail result for each module and time domain. (Replaces tlResultModeSequential used in earlier releases.) IG-XL uses a separate test number for each module in each time domain.

Usage

This parameter not only affects the datalog output, but it can also affect the number of pattern bursts executed. If using the tlResultModeModule option for the ResultMode parameter on a pattern set with the burst property set to yes, then IG-XL performs a test on all sites in parallel as a single burst as long as there are no failing sites or only one failing site. However, if more than one failing site occurs, then IG-XL runs the pattern modules serially on each site for all sites, whether or not they failed, which results in a longer test execution time that increases proportionally with the number of tested sites.

The `tlResultModeModule` option also affects how IG-XL reports results. If you have multiple modules in a pattern start, then IG-XL bursts each module individually and only reports the result from the last burst module. If the test has more than one failing site, then IG-XL bursts the pattern for each site and provides a result for the last burst site only. Given this behavior, do not run characterization on a multisite test that bursts a pattern with this option enabled. IG-XL returns an error for this setup because results from all sites are not available to the characterization tools.

Related Topics

- [Burst Column](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlFunctional.6.09

<

>

Functional VBT Test : Functional VBT Test Parameters : ConcurrentMode

ConcurrentMode

Specifies the concurrent mode to be used when executing a pattern.

Valid Values

Values are available from a drop-down list:

tlPatConcurrentModeCached	Cache the name of the pattern and inform the flow that flow steps on other concurrent subflows can be executed. Do not start the specified pattern. The flow will start all cached patterns synchronously after determining whether each concurrent subflow has a pattern to be started synchronously. Default.
tlPatConcurrentModeImmediate	Start the pattern immediately and inform the flow that flow steps on other concurrent subflows can be executed. Flow steps on other concurrent flows can be executed concurrently with pattern execution.
tlPatConcurrentModeNone	Concurrency is disabled. Start the pattern immediately but do not allow execution of flow steps on other concurrent subflows.
tlPatConcurrentModeDefault	Use concurrent test mode specified by: TheExec.Flow.Concurrent. VBTPatternsLanguageDefault.

Usage

This parameter specifies whether patterns started should enable concurrent tests to be executed and whether the patterns should be started immediately or cached for subsequent synchronous start when concurrency is enabled.

The default value of the parameter is `tlPatConcurrentModeCached`, which is the default value of the `TheHdw.Patterns.Test VBT` statement. This parameter is optional. When the instance editor is launched, no value is specified by default. The default value is specified in the Functional VBT test code.

This parameter has no effect on Functional VBT test parameters executed outside of concurrent flows.

Note that concurrency is disabled when executing any of the following:

- A preconditioning pattern in any VBT test instance
- A hold-state pattern in any VBT test instance
- Patterns in MTO Memory VBT test instances

Related Topics

For more information, see:

- Hold State Pattern Parameters
- Precondition Pattern Parameters

tlFunctional.6.10

<	>
-----------------	-----------------

[Functional VBT Test](#) : [Functional VBT Test Parameters](#) : WaitTimeDomain

WaitTimeDomain

Specifies the time domains that IG-XL will wait for based on the CPU wait flags states specified in the [WaitFlagA](#), [WaitFlagB](#), [WaitFlagC](#), and [WaitFlagD](#), parameters. If you specify a comma separated list of time domains, then the match condition for all domains must occur before the IG-XL executes the flag match interpose function.

Valid Values

Specify a valid time domain name or a comma-separated list of time domain names in the Value field. The default value is an empty string, which means use the System time domain, which is one time domain that represents all pins in the test system.

Related Topics

For more information, see:

- | | |
|---|---|
| • | Hold State Pattern Parameters |
| • | Precondition Pattern Parameters |

<	>
-----------------	-----------------

tlMtoMemory.7.01

<

>

MTO Memory VBT Test

MTO Memory VBT Test

The MTO Memory VBT Test programs elements of the MTO instrument. This VBT test type allows you to do the following:

- Set up the MTO for data capture, including the following parameters:
 - Patterns, pattern threading, and pattern timeout
 - Pass/fail results reporting
 - Relay switching mode
 - Address scrambling
 - Resource map
 - Capture setup bits (data strings composed of unscrambled and scrambled address bits, data generator output bits, MTO serializer output bits, and logic 0 and 1 bits)
 - Capture conditions (all cycles, all failing cycles, all vectors containing stv, or none)
- Set up the address counters, including the following parameters:

- [Preset values for the X, Y, and Z databus registers](#)

- [Values for the X, Y, and Z address counter registers](#)

- [16-bit X, Y, and Z address enable masks](#)

- [Specify the data generator setup and 16-bit coincidence enable mask](#)

- [Set up the X, Y, and Z scramble algorithms and topological inversion algorithms](#)

- [Set up memory fail processing \(MFP\), including redundancy analysis \(RA\), fail RAM \(FRAM\), and bitmap file creation](#)

For a list of parameters used in this VBT test, see [MTO Memory VBT Test Parameters](#).

For information on the other groups of parameters used in this VBT test, see:

- [Interpose Functions Parameters](#)

- [Flag Match Parameters](#)

- [Pin States Parameters](#)

- [Timing and Levels Tab](#)

- [Limits Tab](#)

For more information, see:

- [About the MTO](#)

- [MTO Theory of Operation.](#)

- [MTO Programming](#)

- [Memory Fail Processing](#)

- [MTO](#) (VBT Syntax)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.02

<

>

MTO Memory VBT Test : MTO Memory VBT Test Parameters

MTO Memory VBT Test Parameters

The MTO Memory VBT Test parameters are summarized below. Click a link for details. These parameters are divided into the following groups:

- Pattern and Control Parameters
- MTO Setup Parameters
- Address Preset Parameters
- Address Gen Parameters
- Algorithms Parameters
- MFP Parameters

Pattern and Control Parameters

These required parameters set up MTO patterns, relay mode, and how to return results.

Patterns	The patterns, pattern sets, and pattern groups to execute.
PatThreading	Enables pattern threading, which is the capability of executing patterns independently at each site.
RelayMode	Operational switching mode: unpowered (default) or powered (hot).

PatternTimeout	The timeout period for pattern execution.
ReportResult	Whether to return the pass/fail result.

MTO Setup Parameters

These required parameters set up the MTO for data capture.

ScrambleAddresses	Whether address scrambling is enabled.
ResourceMap	The resource map to be used to map MTO resources to DUT pins.
CapA, CapB, CapC, CapD	The unscrambled and scrambled address bits, data generator output bits, MTO serializer output bits, and logic 0 and 1 bits to capture on the row and column for use with topological correction and bitmapping.
CaptureMark	The capture conditions for which pattern data is to be captured in the capture memory.

Address Preset Parameters

These parameters allow you to set up the preset values for the X, Y, and Z databus counter registers.

X0, X1, X2, X3, X4, X5, X6, X7, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7, Z0, Z1, Z2, Z3, Z4, Z5, Z6, Z7	A preset value that can be loaded into each X, Y, and Z databus counter register.
--	---

Address Gen Parameters

These parameters allow you to set up the values for the X, Y, and Z address counter registers and enable masks.

Xa, Xb, Xc, Xd, Ya, Yb, Yc, Yd, Za, Zb, Zc, Zd	A value that can be loaded into each X, Y, and Z address counter register.
XEnableMask, YEnableMask, ZEnableMask	The 16-bit X, Y, and Z enable masks used by the corresponding address generators to mask unused bits during arithmetic and logical operations.

Algorithms Parameters

These parameters specify the data generator setup, the coincidence enable mask, the X, Y, and Z scramble algorithms, and topological inversion algorithms.

DG	The setup name used to configure the data generator.
DGCoin	The 16-bit address generator enable mask used to indicate the bits that are used if coincidence is specified as the source in the data set.
XScrambleAlg, YScrambleAlg, ZScrambleAlg	The name of the scramble algorithm to be used to program the X, Y, and Z scramble RAMs.
TopoInversionAlg	The name of the topological inversion algorithm to be used to program the topological data inversion RAM.

MFP Parameters

These parameters set up memory fail processing (MFP), including redundancy analysis (RA), fail RAM (FRAM), and bitmap file creation.

RATopology	The topology to be used for RA.
BitmapTopology	The topology to be used for bitmapping.
RAAccum	Whether fail accumulation is in use for RA.
BitmapAccum	Whether fail accumulation is in use for bitmapping.
RAEnable	Whether redundancy analysis is to be performed.
RAAlgFile	Algorithm file to use for RA solution and FRAM file generation.
RARegFile	RA region file to use for RA solution and FRAM file generation.
RASolutionFile	The RA solution filename in which to save RA repair solutions.
FailRAMEnable	Whether an RA FRAM file of fail data is to be generated.
FailRAMFile	The Fail RAM filename where the fail data is to be stored for use with RA offline.
BitmapEnable	Whether a bitmap file is to be generated.
BitmapFormat	The bitmap file format used when generating the bitmap file.
BitmapFile	The bitmap filename in which to save the generated bitmap.

RAAccum	Whether fails are read from capture memory or through a make fails file.
FailsFile	The make fails file specifying a generated fail data pattern.
MaxFRAMSiteFails	The number of fails that are written to the FRAM buffer used for RA and FRAM files.
BitmapFailingSitesOnly	Whether to generate a bitmap files only on the sites that have failures.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.03

	
---	---

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : Patterns

Patterns

The patterns and pattern sets to execute for MTO testing. Note that pattern groups are not supported in UltraFLEX.

Valid Values

Specify a valid pattern or a comma-separated list of patterns in the Value field or select patterns from the Pattern Selector window. No default value.

Usage

You must specify at least one pattern to be executed. The order of the patterns in the list determines the order in which they are executed. You can specify either an uncompressed (.pat) or a compressed (.pat.gz) pattern file.

Click the ellipsis button (...) to bring up the Pattern Selector data box to select patterns. You can use the Disk Patterns control to select a pattern on a disk or the Pattern Groups and Sets control to choose pattern sets defined in the workbook.

This VBT test supports multiple patterns in the same test instance because IG-XL executes each pattern individually. If you are using a pattern file instruments statement (an MTO pattern key) to define the resource map, receive data, serial capture mode, and serial capture pin parameters, each of the patterns in the same test instance can have a different instruments statement (that is, a different pattern key).

Note that concurrent test is disabled when executing a pattern in an MTO Memory VBT test instance. For more information, see [concurrent test link](#).

Concurrent Patterns

To specify patterns that are to run concurrently, use the plus sign (+) to join them. For example, the following specifies that Pat1, Pat2, and Pat3 are to be executed concurrently:

Pat1+Pat2+Pat3

Related Topics

- [Pattern Selector](#)
-
- [Pattern Threading and MTD Patterns](#)
-
- [PatThreading](#)
-
- [PatternTimeout](#)

tlMtoMemory.7.04

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : PatThreading

PatThreading

Enables pattern threading, which is the capability of executing patterns independently at each site.

Valid Values

Values are available from a drop-down list:

False	Disables pattern threading. Multiple sites can share a single pattern generator, with the result that the sites must be executing the same pattern vectors at the same time. Default.
True	Enables pattern threading. Each site’s pattern is executed independently and able to make its own branch decisions regardless of the results of execution at other sites.

Related Topics

- [Pattern Threading and MTD Patterns](#)
- [Patterns](#)
- [PatternTimeout](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.05

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RelayMode

RelayMode

Operational switching mode of the instrument. Determines if instruments and power supplies should be powered up during PreBody execution.

Valid Values

Values are available from a drop-down list:

tlUnpowered	Unpowered relay signal path (cold switching). Default.
tlPowered	Powered relay signal path (hot switching).

Usage

By default, an instrument’s forcing function and DUT power are turned off when relay switching tasks are performed. You can cause the forcing function and DUT power to remain on by setting this argument to tlPowered, thereby enabling hot switching.

Using powered relay mode can result in a moderate performance improvement. Note, however, that hot switching can result in premature failure of relays.

<

>

2015 Teradyne, Inc. All rights reserved.

<

>

PatternTimeout

Valid Values

Related Topics

- | | |
|---|------------------------------------|
| • | Patterns |
| • | PatThreading |
| • | Pattern Threading and MTD Patterns |

tlMtoMemory.7.07

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : ReportResult

ReportResult

Whether the pass/fail result is to be returned to the flow controller and the datalogger.

Valid Values

Values are available from a drop-down list:

pfAlways	Always returns results to the flow controller and datalogger. Default.
pfNever	Never returns results to the flow controller and datalogger.
pfFailsOnly	Returns only failing tests to the flow controller and datalogger.
pfAlwaysOld	Same as pfAlways. Added for backward compatibility. Do not select for new tests.

Usage

This parameter specifies whether to set the pass/fail status of the test, as well as whether to datalog the result. If you set this to pfNever, a failing test is not returned to the flow controller and therefore cannot cause the device to bin out. The default is pfAlways, which always returns the pass/fail result to the flow controller.

<

>

tlMtoMemory.7.08

<
>

MTO Memory VBT Test : MTO Memory VBT Test Parameters : ScrambleAddresses

ScrambleAddresses

Whether address scrambling is enabled.

Valid Values

Values are available from a drop-down list:

False	Disable address scrambling. Default.
True	Enable address scrambling.

Related Topics

- Address Scramblers
-
- Programming the MTO Algorithms

<
>

tlMtoMemory.7.09

<
>

MTO Memory VBT Test : MTO Memory VBT Test Parameters : ResourceMap

ResourceMap

The name of the resource map to be used to map MTO resources to DUT pins. The MTO resource map controls ADB setup, channel configuration, and capture memory reading.

Valid Values

The menu contains the resource map names currently defined on the MTO Resource Map sheet. No default.

Usage

If you define the `MtoResourceMap` parameter in the instruments statement of an active MTO pattern key, IG-XL ignores any resource map name defined in this VBT test instance. To use this VBT test parameter's resource map name, make sure that the instrument statement's `MtoResourceMap` parameter uses its default value, which is an empty string (`""`).

Related Topics

- Programming the MTO Resource Map

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.10

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : CapA, CapB, CapC, CapD

CapA, CapB, CapC, CapD

The unscrambled and scrambled address bits, data generator output bits, MTO serializer output bits, and logic 0 and 1 bits to capture on the row and column for use with topological correction and bitmapping.

Valid Values

Enter data strings in the [CapA](#) (DataString1), [CapB](#) (DataString2), [CapC](#) (DataString3), or [CapD](#) (DataString4) fields. These data strings specify the data to be captured into the MTO capture memory. CapA represents the row address, and CapB represents the column address for bitmapping. CapC and CapD are optional.

The data strings are composed of the following elements:

- Unscrambled address bits:

x0, x1, x2, x3, x4, x5 ,x6, x7, x8, x9, x10, x11, x12, x13, x14, x15
y0, y1, y2, y3, y4, y5 ,y6, y7, y8, y9, y10, y11, y12, y13, y14, y15
z0, z1, z2, z3, z4, z5 ,z6, z7, z8, z9, z10, z11, z12, z13, z14, z15
- Scrambled address bits:

sx0, sx1, sx2, sx3, sx4, sx5 ,sx6, sx7, sx8, sx9, sx10, sx11, sx12, sx13, sx14, sx15
sy0, sy1, sy2, sy3, sy4, sy5 ,sy6, sy7, sy8, sy9, sy10, sy11, sy12, sy13, sy14, sy15
sz0, sz1, sz2, sz3, sz4, sz5 ,sz6, sz7, sz8, sz9, sz10, sz11, sz12, sz13, sz14, sz15
- Data generator output bits dgen0 and dgen1.

- [MTO serializer output bits](#) ser0 and ser1.

- [Logic 0 and 1 bits](#) force0 and force1.

Each string can describe up to 16 bits of data to be captured. Strings can include comma separated lists and ranges:

- [Range examples](#)

[x0:x15](#)

[sx15:sx0](#)

- [List examples](#)

[x0, x2, y0, y2, z0, z2](#)

- [Combination examples](#)

[x0:x7, y3:y0, z2, z4, z6, z8](#)

Usage

In [normal usage, only](#) DataString1, which represents the row address, and DataString2, which represents the column address, are specified for bitmapping. If DataString3 and DataString4 are specified, the available depth of capture memory is cut in half. The capture memory depth is normally configured as 64 M x 32 bits of MTO data. So, if you need to store 33 to 64 bits of MTO data, then you only have 32 M available. Additionally, software overrides previous data field settings to store in MTO CMEM in this mode.

[IG-XL does not support the](#) DataString3 and DataString4 parameters in single timing mode and ignores them. In dual timing mode, these parameters are optional. If they are not specified for dual timing mode, IG-XL repeats the address specified by DataString1 and DataString2 for odd cycles. If they are specified for dual timing mode, IG-XL uses the address specified by DataString1 and DataString2 for even cycles and the address specified by DataString3 and DataString4 for odd cycles.

[When bitmapping in single 1X mode,](#) DataString1 and DataString2 represent row and column address, respectively. When bitmapping in dual 1X mode, DataString1 and DataString2 represent row and column address for even cycles, and DataString3 and DataString4 represent row and column address for odd cycles.

[To visualize how a data string compiles, read it backwards from right to left, filling in instructions from LSB to MSB as you proceed. For example, the data string](#) "sy4:sy0, z2" compiles to the

following one block of 16 ADS instructions: location 0 = z2, location 1 = sy0, location 2 = sy1, location 3 = sy2, location 4 = sy3, and location 5 = sy4.

Related Topics

- [MTO Capture Memory](#)
- [Capturing Fail Data](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlMtoMemory.7.11

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : CaptureMark

CaptureMark

The capture conditions for which pattern data is to be captured in the capture memory.

Valid Values

Values are available from a drop-down list:

CmemCaptAll	Capture on all cycles beginning with pattern start. Default.
CmemCaptFail	Capture on all failing cycles.
CmemCaptSTV	Capture on all vectors that contain the stv microcode. (STV means "store this vector.")
CmemCaptNone	Do not capture.

Related Topics

- [MTO Capture Memory](#)
- [Capturing Fail Data](#)

<

>

tlMtoMemory.7.12

<	>
---	---

MTO Memory VBT Test : MTO Memory VBT Test Parameters : X0, X1, X2, X3, X4, X5, X6, X7, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7, Z0, Z1, Z2, Z3, Z4, Z5, Z6, Z7

X0, X1, X2, X3, X4, X5, X6, X7, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7, Z0, Z1, Z2, Z3, Z4, Z5, Z6, Z7

A preset value that can be loaded into each X, Y, and Z databus counter register. Specify a separate preset value for each X, Y, and Z databus counter register.

Valid Values

Enter a hexadecimal or decimal value up to 65535 (hex = &HFFFF). You can also enter an equation. Default is 65535 (&HFFFF).

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Address Presets Example Pattern File

You can load the address counter register preset values from VBT test instance values or from within a pattern. The pattern syntax for these two methods differs. When loading preset values from the VBT test instance, you are loading 16 bits of data from one of the eight databus registers. When loading preset values from the SRM, you must indicate this in the pattern file with the `sv` syntax and set the actual preset value for each X, Y, and Z SRM register (0-15). SRM preset values cannot be loaded from the address presets template fields. The default is to load pattern register values from the VBT test instance. The `xreg`, `yreg`, and `zreg` pattern values specify which of the preset registers to use. The following syntax shows how to do this:

```
import tset read;
instruments = { mto; }
```

```
srm_vector ($tset  ABUS:S ABUSLOOP:H )  
{  
(mto= xreg 1  xa load_reg  xb load_reg  xc load_reg  xd load_reg)  
stv  > read  .D  .Rf ; // load xa, xb, xc, xd with the x1 address preset  
}
```

In each vector, the pattern can specify loading preset values from the VBT test instance or from within the pattern itself. The following syntax shows how to load SRM address preset values from within a pattern:

```
(mto= xreg 2  yreg 2  xpreset_select sv  ypreset_select sv  
      xa jam_reg  ya jam_reg)  
stv  > read  .D  .Rf ; // load xa and ya registers with the value 2
```

Related Topics

- | | |
|---|--|
| • | Xa, Xb, Xc, Xd, Ya, Yb, Yc, Yd, Za, Zb, Zc, Zd |
| • | Expression Builder |
| • | Address Generators |
| • | Address Counters |
| • | XCounter |
| • | XCounterPreset |
| • | YCounter |
| • | YCounterPreset |
| • | ZCounter |
| • | ZCounterPreset |

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tlMtoMemory.7.13

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : Xa, Xb, Xc, Xd, Ya, Yb, Yc, Yd, Za, Zb, Zc, Zd

Xa, Xb, Xc, Xd, Ya, Yb, Yc, Yd, Za, Zb, Zc, Zd

A value that can be loaded into each X, Y, and Z address counter register. Specify a separate X, Y, and Z value for each address counter register.

Valid Values

Enter a decimal or hexadecimal value up to 65535 (hex = &HFFFF). No default.

Related Topics

- [X0, X1, X2, X3, X4, X5, X6, X7, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7, Z0, Z1, Z2, Z3, Z4, Z5, Z6, Z7](#)
- [Address Generators](#)
- [Address Counters](#)
- [XCounter](#)
- [XCounterPreset](#)
- [YCounter](#)
- [YCounterPreset](#)
- [ZCounter](#)

•	ZCounterPreset
---	--------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.14

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : XEnableMask, YEnableMask, ZEnableMask

XEnableMask, YEnableMask, ZEnableMask

The 16-bit X, Y, and Z enable masks used by the corresponding address generators to mask unused bits during arithmetic and logical operations. For linked operations, you can set the overflow bit after incrementing past the maximum value enabled by the mask. A 1 enables the bit, and a 0 masks the bit.

Valid Values

Enter a decimal or hexadecimal value up to 65535 (hex = &HFFFF) or an equation. Default is 65535 (&HFFFF).

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- X0, X1, X2, X3, X4, X5, X6, X7, Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7, Z0, Z1, Z2, Z3, Z4, Z5, Z6, Z7
- Xa, Xb, Xc, Xd, Ya, Yb, Yc, Yd, Za, Zb, Zc, Zd
- Expression Builder

- [Address Generators](#)

- [Address Counters](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.15

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : DG

DG

The setup name used to configure the data generator.

Valid Values

The menu contains the setup names currently defined on the MTO Data Generator sheet. No default.

Related Topics

- [DGCoin](#)
- [Data Generators](#)
- [Programming the MTO Data Generator](#)

<	>
-----------------	-----------------

tlMtoMemory.7.16

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : DGCoin

DGCoin

The 16-bit address generator enable mask used to indicate the bits that are used if coincidence is specified as the source in the data set. A 1 bit in the mask indicates that the X and Y address bits at this position are to be compared.

The X, Y, and Z enable masks do not have to contain the same enable bits, but they should contain the same number of enabled bits.

Valid Values

Enter a decimal or hexadecimal value up to 65535 (hex = &HFFFF) or an equation. Default is 0.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [DG](#)
- [Expression Builder](#)
- [Data Generators](#)
- [Programming the MTO Data Generator](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlMtoMemory.7.17

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : XScrambleAlg, YScrambleAlg, ZScrambleAlg

XScrambleAlg, YScrambleAlg, ZScrambleAlg

The name of the scramble algorithm to be used to program the X, Y, and Z scramble RAMs. Specify a separate algorithm for the X, Y, and Z address.

Valid Values

The menu contains the algorithm names of type Scramble currently defined on the MTO Algorithm sheet. No default.

Related Topics

- | | |
|---|--|
| • | Address Scramblers |
| • | Programming the MTO Algorithms |
| • | BypassScramblingToDut |
| • | ProgramScrambleRamsByName |

<	>
-----------------	-----------------

tlMtoMemory.7.18

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : TopoInversionAlg

TopoInversionAlg

The name of the topological inversion algorithm to be used to program the topological data inversion RAM.

Valid Values

The menu contains the algorithm names of type Topo Data Invert currently defined on the MTO Algorithm sheet. No default.

Related Topics

- | | |
|---|--|
| • | Topological Data Inversion |
| • | Programming the MTO Algorithms |
| • | Topological Correction |
| • | BypassTopoInvertToDut |
| • | ProgramTopoRamByName |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.19

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RATopology

RATopology

The topology specified on the Memory Topology sheet to use for topologically correcting the fail data for use in redundancy analysis (RA). The topology maps physical fail addresses and their corresponding I/O pin fails onto a topologically correct bit image. This topology is used for both RA and FRAM file generation.

Valid Values

From the menu, which contains the topologies defined on the program workbook's Memory Topology sheet. By default, no topology is selected.

Usage

For RA solutions to be generated, the test instance must specify the RA topology, the solution filename, the region file, and the algorithm file.

For Fail RAM files to be produced, the test instance must specify the RA topology, the FRAM filename, the region file, and the algorithm file.

Related Topics

- | | |
|---|--------------------------------|
| • | RAAccum |
| • | RAEnable |
| • | RAAlgFile |
| • | RARegFile |
| • | RASolutionFile |

- [MaxFRAMSiteFails](#)
- [Topological Correction](#)
- [Redundancy Analysis](#)
- [Initialize](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlMtoMemory.7.20

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [BitmapTopology](#)

BitmapTopology

Which of the topologies defined on the Memory Topology sheet is to be used for topologically correcting the fail data for use in creating a bitmap file. The topology maps physical fail addresses and their corresponding I/O pin fails onto a topologically correct bit image.

Valid Values

Specify from the menu, which contains the topologies defined on the program workbook’s Memory Topology sheet. By default, no topology is selected.

Related Topics

- | | |
|---|---|
| • | BitmapAccum |
| • | BitmapEnable |
| • | BitmapFormat |
| • | BitmapFile |
| • | BitmapFailingSitesOnly |
| • | Topological Correction |
| • | Production Bitmap (PBM) |

- [MTO Bitmap Tool](#)
-
- [Initialize](#)
-
- [SetBitmapFile](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlMtoMemory.7.21

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RAAccum

RAAccum

Whether RA accumulation is to be performed. Accumulation allows fails to be collected across multiple instances.

Valid Values

Values are available from a drop-down list:

AccumNone	Do not perform fail accumulation. Default. Every test that uses this option is a new test in that MFP clears any previous fail data and creates a new bit image if FRAMEnable or RAEnable is set to True.
AccumStart	Begin accumulating fails. Use this option if you are collecting data directly into capture memory and the instance you are using is the first instance you are collecting data on. When accumulation begins, an internal bit image is created to hold the fails. The fails are read from capture memory after the pattern runs, and bitmapping and RA is performed if requested. A bit image is always built, whether or not FRAMEnable or RAEnable is set to True.
AccumContinue	Logically OR the fails found in the current instance with previous fails. Use this option if you are collecting data directly into the capture memory and the instance you are using is not the first instance you are collecting data on. For each instance that specifies accumulation to continue, the fails are read from capture memory and OR-ed into the bit image. A bit image is

always accumulated, whether or not FRAMEEnable or RAEnable is set to True.

Related Topics

• [RATopology](#)

• [RAEnable](#)

• [RAAlgFile](#)

• [RARegFile](#)

• [RASolutionFile](#)

• [MaxFRAMSiteFails](#)

• [Redundancy Analysis](#)

• [Initialize](#)

tlMtoMemory.7.22

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [BitmapAccum](#)

BitmapAccum

Whether bitmap accumulation is to be performed. Accumulation allows fails to be collected across multiple instances.

Valid Values

Values are available from a drop-down list:

AccumNone	Do not perform fail accumulation. Default. Every test that uses this option is a new test in that MFP clears any previous fail data and creates a new bit image if <code>BitmapEnable</code> is set to <code>True</code>.
AccumStart	Begin accumulating fails. Use this option if you are collecting data directly into capture memory and the instance you are using is the first instance you are collecting data on. When accumulation begins, an internal bit image is created to hold the fails. The fails are read from capture memory after the pattern runs, and bitmapping and RA is performed if requested. A bit image is always built, whether or not <code>BitmapEnable</code> is set to <code>True</code>.
AccumContinue	Logically OR the fails found in the current instance with previous fails. Use this option if you are collecting data directly into the capture memory and the instance you are using is not the first instance you are collecting data on. For each instance that specifies accumulation to continue, the fails are read from capture memory and OR-ed into the bit image. A bit image is

always accumulated, whether or not BitmapEnable is set to True.

Related Topics

- | | |
|---|---|
| • | BitmapTopology |
| • | BitmapEnable |
| • | BitmapFormat |
| • | BitmapFile |
| • | BitmapFailingSitesOnly |
| • | Production Bitmap (PBM) |
| • | MTO Bitmap Tool |
| • | Initialize |
| • | SetBitmapFile |

tlMtoMemory.7.23

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RAEnable

RAEnable

Whether redundancy analysis is to be performed. When selected, RA executes, generating repair solutions. Redundancy analysis is performed on the topologically corrected fails as determined by the RA topology.

Valid Values

Values are available from a drop-down list:

False	RA is not performed. Default.
True	RA is performed.

Usage

For RA solutions to be generated, the test instance must specify the RA topology, the solution filename, the region file, and the algorithm file.

Related Topics

- [RATopology](#)
- [RAAccum](#)
- [RAAlgFile](#)
- [RARegFile](#)

- [RASolutionFile](#)
-
- [MaxFRAMSiteFails](#)
-
- [Redundancy Analysis](#)
-
- [Initialize](#)





2015 Teradyne, Inc. All rights reserved.

tlMtoMemory.7.24

<	>
-----------------	-----------------

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [RAAlgFile](#)

RAAlgFile

The RA algorithm file to use for RA solution and FRAM file generation. The algorithm file specifies which RA algorithm to use, such as All, Best, Most, or RA/Plus. It also describes other RA options, such as processing limits, post-processing functions, and fail bit analysis (FBA).

Valid Values

Enter the file path of the RA algorithm file into the text box or bring up the browser to navigate to the file. No default.

Usage

For RA solutions to be generated, the test instance must specify the RA topology, the solution filename, the region file, and the algorithm file.

For Fail RAM files to be produced, the test instance must specify the RA topology, the FRAM filename, the region file, and the algorithm file.

Related Topics

- | | |
|---|--------------------------------|
| • | RATopology |
| • | RAAccum |
| • | RAEnable |
| • | RARegFile |
| • | RASolutionFile |

•	FailRAMEnable
•	FailRAMFile
•	MaxFRAMSiteFails
•	Redundancy Analysis
•	Repair Algorithms
•	Initialize

tlMtoMemory.7.25

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RAREgFile

RAREgFile

The RA region file to use for RA solution and FRAM file generation. The region file defines the device in terms of its redundant elements, providing information about the RA regions and spares.

Valid Values

Enter the file path of the RA region file in the text box or bring up the browser to navigate to the file. No default.

Usage

For RA solutions to be generated, the test instance must specify the RA topology, the solution filename, the region file, and the algorithm file.

For Fail RAM files to be produced, the test instance must specify the RA topology, the FRAM filename, the region file, and the algorithm file.

Related Topics

- [RATopology](#)
- [RAAccum](#)
- [RAEnable](#)
- [RAAlgFile](#)
- [RASolutionFile](#)
- [FailRAMEnable](#)

- [FailRAMFile](#)
- [MaxFRAMSiteFails](#)
- [Redundancy Analysis](#)
- [Initialize](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlMtoMemory.7.26

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : RASolutionFile

RASolutionFile

The RA solution filename in which to save RA repair solutions.

Valid Values

Enter the RA solution filename in the text box. No default. You can insert filename keywords into the filename to serve as a prototype.

Usage

For RA solutions to be generated, the test instance must specify the RA topology, the solution filename, the region file, and the algorithm file.

RA solution filenames take the .soln suffix.

Filename Keywords

When specifying the RA solution filename, you can use filename keywords. The keywords are filled in with their appropriate value at the time the solution file is written. Each keyword must be within curly braces: for example, {wafer_id}. The following keywords are supported.

- {wafer_id}
- {lot_id}
- {current_site}: Note that unless this keyword is used explicitly, IG-XL automatically appends the site number to the filename.
- {instance}
- {xcoord}

- [{ycoord}](#)
- [{family_id}](#)
- [{flow_id}](#)
- [{test_temperature}](#)
- [{part_id}](#)
- [{message}](#)
- [{process_id}](#)
- [{test_frequency}](#)
- [{design_rev}](#)

Related Topics

- [RATopology](#)
- [RAAccum](#)
- [RAEnable](#)
- [RAAlgFile](#)
- [RARegFile](#)
- [MaxFRAMSiteFails](#)
- [Redundancy Analysis](#)

- [Initialize](#)

- [SetSolutionFile](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.27

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : FailRAMEnable

FailRAMEnable

Whether an RA Fail RAM file of fail data is to be generated. The FRAM file is a representation of the fails captured in a form suitable for use by RA.

Valid Values

Values are available from a drop-down list:

False	FRAM file is not generated. Default.
True	FRAM file is generated.

Usage

For Fail RAM files to be produced, the test instance must specify the RA topology, the FRAM filename, the region file, and the algorithm file.

Related Topics

- [RATopology](#)
- [RAAlgFile](#)
- [RARegFile](#)
- [FailRAMFile](#)
- [MaxFRAMSiteFails](#)

- [Redundancy Analysis](#)

- [Initialize](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.28

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : FailRAMFile

FailRAMFile

The Fail RAM filename where the fail data is to be stored for use with RA offline. The FRAM file contains the fail data broken down by region as topologically corrected by the RA topology.

Valid Values

Enter the FRAM filename in the text box. No default. You can insert filename keywords into the filename to serve as a prototype.

Usage

For Fail RAM files to be produced, the test instance must specify the RA topology, the FRAM filename, the region file, and the algorithm file.

Filename Keywords

When specifying a FRAM filename, you can use filename keywords. The keywords are filled in with their appropriate value at the time the FRAM file is written. Each keyword must be within curly braces: for example, {wafer_id}. The following keywords are supported:

- | | |
|---|--|
| • | {wafer_id} |
| • | {lot_id} |
| • | {current_site}: Note that unless this keyword is used explicitly, IG-XL automatically appends the site number to the filename. |
| • | {instance} |
| • | {xcoord} |

- [{ycoord}](#)

- [{family_id}](#)

- [{flow_id}](#)

- [{test_temperature}](#)

- [{part_id}](#)

- [{message}](#)

- [{process_id}](#)

- [{test_frequency}](#)

- [{design_rev}](#)

Related Topics

- [RATopology](#)

- [RAAlgFile](#)

- [RARegFile](#)

- [FailRAMEnable](#)

- [MaxFRAMSiteFails](#)

- [Redundancy Analysis](#)

- [Initialize](#)

•	SetFRAMFile
---	-----------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.29

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : BitmapEnable

BitmapEnable

Whether a bitmap file is to be generated. The bitmap file is a topologically corrected representation of the fails.

Valid Values

Values are available from a drop-down list:

False	Bitmap file is not generated. Default.
True	Bitmap file is generated.

Usage

For bitmap files to be produced, the test instance must specify the bitmap topology, the bitmap filename, and the bitmap format.

Related Topics

- [BitmapTopology](#)
- [BitmapAccum](#)
- [BitmapFormat](#)
- [BitmapFile](#)
- [BitmapFailingSitesOnly](#)

•	Production Bitmap (PBM)
•	MTO Bitmap Tool
•	Initialize
•	SetBitmapFile

tlMtoMemory.7.30

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [BitmapFormat](#)

BitmapFormat

The bitmap file format used when generating the bitmap file.

Valid Values

Values are available from a drop-down list:

mfpBitmapBMDB	Binary database format that stores both passing and failing bits for every cell in a single device. File extension is .bmdb . Default.
mfpBitmapBMAP	ASCII text format that stores each failing address in a single device. File extension is .bmap .
mfpBitmapMFD	Memory fail data format that stores complete bit images for the device and also contains topology information used to create the data. This is the recommended file storage method for MTO. File extension is .mfd .
mfpBitmapPBM	Production Bitmap (PBM) format. File extension is .pbm .
mfpBitmapPNG	Portable Network Graphics format. File extension is .png . Not supported.

Usage

For bitmap files to be produced, the test instance must specify the bitmap topology, the bitmap filename, and the bitmap format.

Related Topics

- [BitmapTopology](#)
- [BitmapAccum](#)

•	BitmapEnable
•	BitmapFile
•	BitmapFailingSitesOnly
•	Production Bitmap (PBM)
•	MTO Bitmap Tool
•	Bitmap File Formats
•	Initialize
•	SetBitmapFile

tlMtoMemory.7.31

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [BitmapFile](#)

BitmapFile

The bitmap filename in which to save the generated bitmap.

Valid Values

Enter the bitmap filename in the text box. No default. You can insert filename keywords into the filename to serve as a prototype.

Usage

For bitmap files to be produced, the test instance must specify the bitmap topology, the bitmap filename, and the bitmap format.

Filename Keywords

When specifying a bitmap filename, you can use filename keywords. The keywords are filled in with their appropriate value when the bitmap file is written. Each keyword must be within curly braces: for example, {wafer_id}. The following keywords are supported:

- {wafer_id}
- {lot_id}
- {current_site}**: Note that unless this keyword is used explicitly, IG-XL automatically appends the site number to the filename.
- {instance}
- {xcoord}

- [{ycoord}](#)

- [{family_id}](#)

- [{flow_id}](#)

- [{test_temperature}](#)

- [{part_id}](#)

- [{message}](#)

- [{process_id}](#)

- [{test_frequency}](#)

- [{design_rev}](#)

Related Topics

- [BitmapTopology](#)

- [BitmapAccum](#)

- [BitmapEnable](#)

- [BitmapFormat](#)

- [BitmapFailingSitesOnly](#)

- [Production Bitmap \(PBM\)](#)

- [MTO Bitmap Tool](#)

- [Initialize](#)

- [SetBitmapFile](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlMtoMemory.7.32

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [GenerateFailsFile](#)

GenerateFailsFile

Whether fails are read from capture memory or through a make fails file.

Valid Values

Values are available from a drop-down list:

False	Capture memory is the source of fails. Default
True	A generated make fails file is the source of fails. This file is in the FailsFile parameter.

Usage

Make fails uses a fails file that specifies each of the fails. When make fails is in use, capture memory is not read.

This feature allows you to generate specific fail data for a given X, Y address and I/O at that address. You can use this feature to debug topology, bitmap, or redundancy analysis setup. Make fails is intended for debugging during test, but not during production.

Related Topics

- [FailsFile](#)
- [Capturing Fail Data](#)
- [GetFailData](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlMtoMemory.7.33

<

>

MTO Memory VBT Test : MTO Memory VBT Test Parameters : FailsFile

FailsFile

The make fails file specifying a generated fail data pattern.

Valid Values

Either enter the filename in the text input field or bring up the browser to navigate to the file.

Usage

The fail data in this file applies to all sites. Make fails is intended for debugging only and substitutes for reading fail data from capture memory.

Make Fails File Syntax

Use the following make fails file syntax formatting rules:

- Comments begin with a # and continue to the end of the line. For example:

This line is ignored.
- Blank lines are ignored.
- The first nonblank or noncomment line must be the word "count" followed by the number of fails contained within the fail file. You can have more fail entries than the count specifies, and any entries beyond the count are ignored. If fewer fail entries are present than the count indicates, IG-XL generates a warning and uses only the fails present.
- After the count line follows the set of fail entries. Only one fail can be specified per line. Each line is of the form:

<y-address> <x-address> <I/O bit> <site number>

The <y-address> and <x-address> are 16-bit numbers. The <I/O bit> is a bit index starting at 0. The MTO Resource Map sheet establishes the index ordering for I/O bits. The <site number> is optional. IG-XL writes a fail to all sites if there is no site number specified.

The following is an example of the make fails file syntax:

```
count 5
0 0 0
0 0 0
1 2 1
0 1 2
2 2 2
```

Related Topics

- [RAAccum](#)
- [Capturing Fail Data](#)
- [GetFailData](#)

<

>

tlMtoMemory.7.34

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : MaxFRAMSiteFails

MaxFRAMSiteFails

The number of fails that are written to the FRAM buffer used for RA and FRAM files.

Valid Values

Enter a numeric value into the text field or an equation. No default.

Usage

This parameter’s value is used to limit the number of fails written to the buffer before aborting RA. It is usually specified as something slightly larger than the maximum number of fails for a repairable device, and it would be determined based on the spares available for repairing the device.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all the spec variables defined on the Specs sheets. You can also manually enter a numeric value or an equation into the data area. Once you have entered an equation, the present value of the equation is displayed to the right of each button displays. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- [RATopology](#)
- [RAAccum](#)
- [RAEnable](#)
- [RAAlgFile](#)

- [RARegFile](#)
- [RASolutionFile](#)
- [FailRAMEnable](#)
- [FailRAMFile](#)
- [MaxFRAMSiteFails](#)
- [Expression Builder](#)
- [Redundancy Analysis](#)
- [Initialize](#)
- [SetSolutionFile](#)

<

>

tlMtoMemory.7.35

<

>

MTO Memory VBT Test : [MTO Memory VBT Test Parameters](#) : [BitmapFailingSitesOnly](#)

BitmapFailingSitesOnly

Whether to generate bitmap files only on the sites that have failures. With this option enabled, MTO skips unneeded data processing on passing sites, improving bitmapping test times.

Valid Values

Values are available from a drop-down list:

False	Generate bitmap files on all sites. Default.
True	Generate bitmap files only on the sites that have failures.

Usage

This optional parameter is used for bitmapping only with .bmdb and .bmap files. A single .mfd file is always created for all sites.

Related Topics

- [BitmapTopology](#)
- [BitmapAccum](#)
- [BitmapEnable](#)
- [BitmapFormat](#)
- [BitmapFile](#)

- [Production Bitmap \(PBM\)](#)
- [MTO Bitmap Tool](#)
- [Initialize](#)
- [SetBitmapFile](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlPinPmu.8.01

<	>
-----------------	-----------------

Pin PMU VBT Test

Pin PMU VBT Test

The Pin PMU VBT test programs elements of a Pin Parametric Measurement Unit (PPMU). This VBT test type allows you to do the following:

- [Select pins to be tested](#)
- [Set voltage or current measurement](#)
- [Force values for first and second executions of the test loop](#)
- [Set a force and measure current range for the PPMU during measurement](#)
- [Set high and low voltage clamps](#)
- [Select an operational switching mode](#)
- [Set the number of samples to take](#)
- [Set the settling time](#)
- [Specify an alternate set of pins with different test conditions and test them in parallel with the primary set of specified pins](#)

- [Specify if a test is to be performed serially rather than in parallel](#)

✓ Note:

You do not program the test limits directly in a Pin PMU VBT test instance. Instead, you program test limits in the Flow Table sheet with the Use-Limit opcode. This allows you to use the same PPMU test instance with different test limits.

Moreover, the Pin PMU VBT test datalog uses only the high and low limit values entered in the Use-Limit row. The datalog takes all other values, including the test name, test number, pass/fail bin numbers, pass/fail sort numbers, and pass/fail result flag, from the Test row.

Finally, the Pin PMU VBT test does not support the Test-defer-limits opcode.

For more information, see [Test Limits in Flow](#).

✓ Note:

When using alternate test conditions, you must include a second Use-Limit row for the same test instance. You can use this row to specify an alternate set of test limits. An alternate set of test limits is optional, but the additional row is required. If you have multiple test instances that use the same name, then only the ones using alternate test conditions require an additional row.

IG-XL returns a run-time error if you specify pins in the PinsAlt parameter but do not include an additional Use-Limit row in the Flow Table for the test instance with the alternate test conditions.

✓ Note:

When setting a breakpoint on a PPMU test instance in the test flow, the prebody, body, and postbody breakpoints do not stop at a point that have programmed PPMU hardware test conditions. To see the test conditions for debugging purposes, you must open the VB editor and put a breakpoint at the following line in the body section of the PinPMU_T template module:

```
ReturnStatus = tl_PPMUMeasure(Pins, SampleCount, SettlingTime, ConvertedRelayMode)
```

For a list of parameters used in this VBT test, see [Pin PMU VBT Test Parameters](#).

For information on other parameter groups used in this VBT test, see:

- [Hold State Pattern Parameters](#)

- [Interpose Functions Parameters](#)

- [Pin States Parameters](#)

- [Precondition Pattern Parameters](#)
- [Timing and Levels Tab](#)
- [Limits Tab](#)

For more information, see:

- [About the PPMU](#)
- [PPMU Programming](#)
- [PPMU Examples](#)
- [PPMU \(VBT Syntax\)](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlPinPmu.8.02

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters

Pin PMU VBT Test Parameters

Pin PMU VBT Test parameters are summarized below.

Pins	Pins and pin groups to be tested.
MeasureMode	Whether the PPMU is measuring voltage or current.
ForceCond1 and ForceCond2	The voltage or current to be forced on the specified pins. ForceCond1 is the force value for the required first execution of the test loop, and ForceCond2 is the value for an optional second test loop execution.
ForcelRange	Force current range for the PPMU when forcing current and measuring voltage.
MeasIRange	Current measurement range for the PPMU when forcing voltage and measuring current.
VClampLo and VClampHi	The high and low clamp voltages, that is, the voltages at which the PPMU limits current flow.
RelayMode	Operational switching mode of the PPMU: unpowered (cold) or powered (hot).
SettlingTime	The wait time in seconds before the PPMU takes the measurement.
SamplingTime	The total time in seconds during which the PPMU is to take measurement samples.
SampleCount	The number of samples to take and average during test measurement.

The Alternate Test Conditions group of Pin PMU VBT Test parameters are summarized below. These parameters allow you to define another set of pins with different test conditions and test them in parallel with the pins defined in the Pins parameter.

PinsAlt	Pins and pin groups to be tested in parallel with the pins defined in the Pins parameter.
MeasureModeAlt	Whether the PPMU is measuring voltage or current on PinsAlt.
ForceCondAlt	The voltage or current to be forced on PinsAlt.
ForceRangeAlt	Force current range for the PPMU when forcing current and measuring voltage on PinsAlt.
MeasRangeAlt	Measure current range for the PPMU when forcing voltage and measuring current on PinsAlt.

The Testing Pins Serially group of Pin PMU VBT Test parameters are summarized below.

TestingInSeries	Test specified pins serially instead of in parallel.
BackgroundMeasureMode	The PPMU measurement mode for serial testing: force current measure voltage (FIMV) or force voltage measure current (FVMI).
BackgroundForceCond	The voltage or current to be forced on the specified pins for serial testing.
BackgroundForceRange	The PPMU force current (I) range for serial testing.
BackgroundMeasRange	The PPMU measure current (I) range for serial testing.

tlPinPmu.8.03

<	>
-----------------	-----------------

Pin PMU VBT Test : Pin PMU VBT Test Parameters : Pins

Pins

Pin and pin groups to be tested.

Valid Values

Specify valid pins or pin groups. The menu contains any pin on the Pin Map sheet whose type is I/O, Input, Output, or Analog (used for BBAC and VHFAC pins), plus any pin group containing only pins whose type is I/O, Input, Output, or Analog. No default value.

Usage

You must specify a pin list by either entering a comma-separated list of pin or pin group names in the Value data box or selecting pins and pin groups from the Pins Selection data box. Click the ellipsis button (...) to bring up the Pins Selection data box to select pins and pin groups. The Available Pins data box contains the names of all digital I/O and analog pins and pin groups on the Pin Map sheet. The order of the pins in the list determines the order in which the pins are tested.

Related Topics

- | | |
|---|-------------------------|
| • | PinsAlt |
|---|-------------------------|
- | | |
|---|--------------------------------|
| • | Pins Selection |
|---|--------------------------------|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.04

<	>
-----------------	-----------------

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : MeasureMode

MeasureMode

The PPMU operating mode can be set to force current and measure voltage (FIMV) or force voltage and measure current (FVMI).

Valid Values

Values are available from a drop-down list:

tlPPMUForceIMeasureV	Force current and measure voltage (FIMV). Default.
tlPPMUForceVMeasureI	Force voltage and measure current (FVMI).

Related Topics

• Mode

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.05

<

>

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : ForceCond1 and ForceCond2

ForceCond1 and ForceCond2

The voltage or current to be forced on the specified pins. ForceCond1 is the force value for the required first execution of the test loop, and ForceCond2 is the value for an optional second test loop execution.

Valid Values

Enter a force voltage or force current value in the Value field. ForceCond1 must be specified. ForceCond2 is optional.

If MeasureMode is tlPPMUForceIMeasureV, ForceCond values are in amps. If it is tlPPMUForceVMeasureI, the ForceCond values are in volts.

Usage

The following table shows force current and force voltage values for UltraFLEX instruments:

PPMU Force Current and Force Voltage Values		
Instrument	Force Current Values	Force Voltage Values
AWG6G, GigaDig, HSD1000, UltraPin800, UltraPin1600	-50mA to 50mA	-1.0V to 6.0V
BBAC, TurboAC, VHFAC	-2.0mA to 2.0mA	-2.0V to 7.0V
DCIO	-64mA to 64mA	-1.0V to 6.0V
HPM	-30mA to 30mA	-1.0V to 1.5V (for 2mA and 200A range) 0V to 1.5V (for 30mA range¹)

- -0.8V to 1.5V (for 20A range)

SB6G	-50mA to 50mA	-1.0V to 3.6V
UltraPin4000	-30mA to 30mA	

- -1.0V to 1.5V (for 2mA and 200A ranges)
- 0V to 1.5V (for 30mA range²)

- 1 For the 30mA measure current range, the full voltage range is only valid when the load current into the DUT is 0. As the load current increases, this range narrows as described below.
- 2 For the 30mA measure current range, the full voltage range is only valid when the load current into the DUT is 0. As the load current increases, this range narrows as described below.

The test loop is executed once or twice, depending on whether ForceCond2 has a value.

- The ForceCond1 value is used during the first execution of the test loop. A ForceCond1 value is required.
- The ForceCond2 value is used during the second execution of the test loop. A ForceCond2 value is optional. If one is specified, the test loop is executed a second time; otherwise, the loop is executed only once, using the ForceCond1 value.

The use of two force values lets you program leakage tests, which are typically specified under two force conditions. They also reduce both time and wear on test system relays, which must be switched only once for the two sets of tests.

If you have already specified a ForceCond2 value, then you cannot use PinsAlt.

Related Topics

- Pins
- MeasureMode

•	Expression Builder
---	------------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.06

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : ForcelRange

ForcelRange
Force current range for the PPMU when forcing current and measuring voltage (FIMV).
Valid Values
Values are available from a drop-down list (no default):

AWG6G, GigaDig, HSD1000, UltraPin800, UltraPin1600	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200uA, 2mA, Auto
DCIO	8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 32mA, Auto
HPM	20uA, 200uA, 2mA, 30mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage
This parameter is valid when MeasureMode is tlPPMUForceIMeasureV.
The Auto range makes a number of measurements to determine the best range.
When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.
Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).

If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- [MeasureMode](#)

- [MeasIRange](#)

- [Forcel](#)

- [Current](#)

- [ForceCurrentRange](#)

	
---	---

tlPinPmu.8.07

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : MeasIRange

MeasIRange

Current measurement range for the PPMU when forcing voltage and measuring current.

Valid Values

Values are available from a drop-down list (no default):

AWG, GigaDig	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200nA, 2uA, 20uA, 200uA, 2mA, Auto
DCIO	- Non-merged PPMU Ranges: 8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 64mA, Auto - Merged PPMU Ranges: 16uA, 64uA, 256uA, 1mA, 4mA, 16mA, 32mA, 128mA, 256mA, 512mA, 1.024A, 2.048A
HPM	20uA, 200uA, 2mA, 30mA, Auto
HSD1000, UltraPin800, UltraPin1600	2uA, 20uA, 200uA, 2mA, 50mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage

This parameter is valid when MeasureMode is tIPPMUForceVMeasureI.

The Auto range makes a number of measurements to determine the best range.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).

If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- MeasureMode
- ForceIRange
- ForceV
- Voltage
- MeasureCurrentRange

tlPinPmu.8.08

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : VClampLo and VClampHi

VClampLo and VClampHi

The high and low clamp voltages, that is, the voltages at which the PPMU limits current flow.

Valid Values

Enter a value in the Value box. The units are in volts (V).

Usage

The following table shows high and low clamp voltage values for UltraFLEX instruments:

PPMU High and Low Voltage Clamp Values		
Instrument	High Voltage Clamp Range (VClampHi)	Low Voltage Clamp Range (VClampLo)
AWG6G, GigaDig	-1.2V to 6.2V	-1.2V to 6.2V
BBAC, TurboAC	-1.57V to 8.18V	-3.18V to 6.57V
DCIO	-1.3V to 6.3V	-1.3V to 6.3V
HSD1000, UltraPin800 ¹	-2.0V to 6.5V	-2.0V to 6.5V
SB6G	-2.0V to 4.7V	-2.0V to 4.7V
VHFAC	-3.0V to 8.0V	-2.0V to 8.0V
UltraPin1600 ²	-1.6V to 6.5V	-1.6V to 6.5V
HPM and UltraPin4000 ³	-0.8V to 1.5V	-0.8V to 1.5V

¹ VClampLo and VClampHi must be 1.6V apart.

²

VClampLo and VClampHi must be 1.0V apart.

3 HPM and UltraPin4000 voltage clamps are not programmable.

Related Topics

- [SetClampsVHi](#)
- [SetClampsVLo](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlPinPmu.8.09

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : RelayMode

RelayMode

Operational switching mode of the instrument. Determines if instruments and power supplies are powered up.

Valid Values

Values are available from a drop-down list:

tlUnpowered	Unpowered relay sequence (cold switching). Default.
tlPowered	Powered relay sequence (hot switching).
tlUnpoweredPreset	Unpowered relay sequence where IG-XL presets the PPMU to the test conditions before connecting.

Usage

By default, an instrument’s forcing function and DUT power are turned off when relay switching tasks are performed. You can cause the forcing function and DUT power to remain on by setting this argument to tlPowered, thereby enabling hot switching.

Using the tlPowered mode can result in a moderate performance improvement. Note, however, that hot switching can result in premature failure of relays.

Use the tlUnpoweredPreset mode when the DUT is susceptible to voltage or current glitching, as this mode provides the cleanest connection sequence. Note that if you use smaller current ranges, then this mode can result in slower performance. Although it can be used with PPMUs on all instrument board types, this mode is typically used with digital board PPMUs.

☑ **Note:**

If TestingInSeries is True, then this parameter must use tlUnpowered.

☑ **Note:**

UltraPin4000 and HPM instruments do not support a channel connected to the PPMU using tIUnpowered in serial or parallel mode unless the adjacent channel is also connected to the PPMU. Do not run PPMU tests in serial mode on HPM or UltraPin4000 using the Pin PMU VBT test template. If you need to do this type of testing, then you must implement custom VBT code to ensure that adjacent channels are in the same mode (PPMU or PE driver).

 Note:

For most instruments, disconnecting the PPMU does not affect the gate state. For UltraPin4000 and HPM instruments, however, disconnecting the PPMU also gates off the PPMU.

Relay Mode PPMU Test Instance Algorithms

The PPMU test instance uses different algorithms for the PPMU settings. The most significant selection is the relay mode: unpowered or powered. The following tables list the test instance algorithms used for the relay modes.

PPMU Test Instance Pre Body Algorithms for Relay Modes

Unpowered	Powered	Unpowered Preset
Power down pin levels		Power down pin levels
Connect all pins	Connect all pins	Connect all pins
Program Utility Data Bits	Program Utility Data Bits	Program Utility Data Bits
	Apply pin levels	
Load timing	Load timing	Load timing
Disconnect float pins	Disconnect float pins	Disconnect float pins
Set start state and init state for pins	Set start state and init state for pins	Set start state and init state for pins

PPMU Test Instance Body Algorithm for Relay Modes

Unpowered	Powered	Unpowered Preset
Force 0V in maximum current range.		Force 0 A or 0V in programmed mode and current range.
Gate on.		Gate on.
Disconnect measure pins from PE.		Disconnect measure pins from PE.

Connect measure pins to PPMU.		Connect measure pins to PPMU.
Apply pin levels.		Apply pin levels.
Run PreCond pattern if specified.	Run PreCond pattern if specified	Run PreCond pattern if specified.
Run hold state pattern if specified.	Run hold state pattern if specified	Run hold state pattern if specified.
Set up PPMU mode and range. Apply Force Value.	Set up PPMU mode and range. Apply Force Value. Gate on. Disconnect measure pins from PE. Connect measure pins to PPMU.	Apply Force Value.
Take measurement.	Take measurement.	Take measurement.
Force 0V in maximum current range.		Force 0 A or 0V in programmed mode and current range.
Disconnect measure pins from PPMU.	Disconnect measure pins from PPMU.	Disconnect measure pins from PPMU.

PPMU Test Instance Post Body Algorithms for Relay Modes

Unpowered	Powered	Unpowered Preset
Disconnect float pins.	Disconnect float pins.	Disconnect float pins.
Return channels to default start state.	Return channels to default start state.	Return channels to default start state.

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.10

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : SettlingTime

SettlingTime

The wait time in seconds before the PPMU takes the measurement. This pause before measurement allows the force value to settle and stabilize. It is measured from the time the PPMU is gated on to the time the measurement is taken, that is, to the start of the analog-to-digital conversion.

Valid Values

Enter a settling time value in seconds, a single spec variable, or an equation. The maximum settling time is 600 seconds. Default is 0 seconds.

Usage

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

The time required before measurement is usually an attribute of the test system hardware, not of the DUT. However, DIBs with unusually high wiring capacitance, or with lump capacitance, may require a higher-than-normal settling time.

Related Topics

- Expression Builder
- SamplingTime
- SampleCount

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlPinPmu.8.11

<	>
-----------------	-----------------

Pin PMU VBT Test : Pin PMU VBT Test Parameters : SamplingTime

SamplingTime

☒ **Note:**

This parameter is not supported in this release. Use the SampleCount parameter to define your sample size.

The total time in seconds during which the PPMU is to take measurement samples.

Valid Values

Enter a sampling time value in seconds, a single spec variable, or an equation.

Usage

If this value is specified, the number of samples equals the SamplingTime value divided by the test system conversion time. In this way, the sampling can be independent of conversion speed.

If neither SampleCount nor SamplingTime is specified, the VBT test uses a sample count of 1.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- | | |
|---|------------------------------------|
| • | Expression Builder |
| • | SettlingTime |
| • | SampleCount |

< >		
	2015 Teradyne, Inc. All rights reserved.	

tlPinPmu.8.12

<	>
-----------------	-----------------

Pin PMU VBT Test : Pin PMU VBT Test Parameters : SampleCount

SampleCount

The number of samples to take and average during test measurement. These samples (that is, the number of analog-to-digital conversions) are used to obtain the test result.

Valid Values

Enter a sample size value, a single spec variable, or an equation. The maximum sample size value is 10000. No default.

Usage

If you do not specify SampleCount, the VBT test uses a sample count of 1.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Related Topics

- | | |
|---|------------------------------------|
| • | Expression Builder |
|---|------------------------------------|
- | | |
|---|------------------------------|
| • | SettlingTime |
|---|------------------------------|
- | | |
|---|------------------------------|
| • | SamplingTime |
|---|------------------------------|

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlPinPmu.8.13

<	>
---	---

Pin PMU VBT Test : Pin PMU VBT Test Parameters : PinsAlt

PinsAlt

Pin and pin groups to be tested in parallel with pins defined in Pins parameter.

Valid Values

Specify valid pins or pin groups. The menu contains any pin on the Pin Map sheet whose type is I/O, Input, Output, or Analog (used for BBAC and VHFAC pins), plus any pin group containing only pins whose type is I/O, Input, Output, or Analog. No default value.

Usage

PinsAlt must contain the same number of pins listed in the Pins parameter. It cannot contain any pin which is already specified in Pins.

If you have already specified a ForceCond2 value, then you cannot use PinsAlt.

You must specify a pin list by either entering a comma-separated list of pin or pin group names in the Value data box or selecting pins and pin groups from the Pins Selection data box.

Click the ellipsis button (...) to bring up the Pins Selection data box to select pins and pin groups.

The Available Pins data box contains the names of all digital I/O and analog pins and pin groups on the Pin Map sheet.

The order of the pins in the list determines the order in which the pins are tested.

☒ **Note:**

When using alternate test conditions, you must include a second Use-Limit row for the same test instance. You can use this row to specify an alternate set of test limits. An alternate set of test limits is optional, but the additional row is required. If you have multiple test instances that use the same name, then only the ones using alternate test conditions require an additional row.

IG-XL returns a run-time error if you specify pins in the PinsAlt parameter, but do not include an additional Use-Limit row in the Flow Table for the test instance with the alternate test conditions.

Related Topics

•	Pins
---	----------------------

•	Pins Selection
---	--------------------------------

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.14

<

>

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : MeasureModeAlt

MeasureModeAlt

The PPMU operating mode can be set to force current and measure voltage (FIMV) or force voltage and measure current (FVMI) for the alternate set of pins.

Valid Values

Values are available from a drop-down list (no default value):

tlPPMUForceIMeasureV	Force current and measure voltage (FIMV).
tlPPMUForceVMeasureI	Force voltage and measure current (FVMI).

Usage

This parameter is required if [PinsAlt](#) is specified.

Related Topics

- [PinsAlt](#)
- [Mode](#)

<

>

tlPinPmu.8.15

<	>
---	---

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : ForceCondAlt

ForceCondAlt

The voltage or current to be forced on the alternate set of pins.

Valid Values

Enter a force voltage or force current value in the Value field.

If MeasureMode is tlPPMUForceIMeasureV, ForceCondAlt values are in amps. If it is tlPPMUForceVMeasureI, the ForceCondAlt values are in volts.

Usage

This parameter is required if PinsAlt is specified.

See [PPMU Force Current and Force Voltage Values](#) for a list of force current and force voltage values for UltraFLEX instruments:

Related Topics

- [PinsAlt](#)
- [MeasureModeAlt](#)
- [Expression Builder](#)

<	>
---	---

tlPinPmu.8.16

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : ForcelRangeAlt

ForcelRangeAlt

Force current range for the PPMU when forcing current and measuring voltage (FIMV) on the alternate set of pins.

Valid Values

Values are available from a drop-down list (no default):

AWG6G, GigaDig, HSD1000, UltraPin800, UltraPin1600	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200uA, 2mA, Auto
DCIO	8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 32mA, Auto
HPM	20uA, 200uA, 2mA, 30mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage

This parameter is valid when MeasureModeAlt is tlPPMUForcelMeasureV.

The Auto range makes a number of measurements to determine the best range.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).
If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- [PinsAlt](#)
- [MeasureModeAlt](#)
- [MeasIRangeAlt](#)
- [Forcel](#)
- [Current](#)
- [ForceCurrentRange](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlPinPmu.8.17

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : MeasIRangeAlt

MeasIRangeAlt

Current measurement range for the PPMU when forcing voltage and measuring current (FVMI) on the alternate set of pins.

Valid Values

Values are available from a drop-down list (no default):

AWG, GigaDig	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200nA, 2uA, 20uA, 200uA, 2mA, Auto
DCIO	- Non-merged PPMU Ranges: 8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 64mA, Auto - Merged PPMU Ranges: 16uA, 64uA, 256uA, 1mA, 4mA, 16mA, 32mA, 128mA, 256mA, 512mA, 1.024A, 2.048A
HPM	20uA, 200uA, 2mA, 30mA, Auto
HSD1000, UltraPin800, UltraPin1600	2uA, 20uA, 200uA, 2mA, 50mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage

This parameter is valid when MeasureModeAlt is tIPPMUForceVMeasureI.

The Auto range makes a number of measurements to determine the best range.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).

If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- PinsAlt
- MeasureModeAlt
- ForceIRangeAlt
- ForceV
- Voltage
- MeasureCurrentRange

tlPinPmu.8.18

<

>

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : [TestingInSeries](#)

[TestingInSeries](#)

Test specified pins serially instead of in parallel.

[Valid Values](#)

Values are available from a drop-down list:

True	Test pins serially.
False	Test pins in parallel. Default.

[Usage](#)

If this parameter is [True](#), then RelayMode must use [tlUnpowered](#).

[Related Topics](#)

- [RelayMode](#)
- [BackgroundMeasureMode](#)
- [BackgroundForceCond](#)
- [BackgroundForceIRange](#)
- [BackgroundMeasIRange](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tlPinPmu.8.19

<

>

Pin PMU VBT Test : [Pin PMU VBT Test Parameters](#) : BackgroundMeasureMode

BackgroundMeasureMode

The PPMU measurement mode for serial testing: force current measure voltage (FIMV) or force voltage measure current (FVMI).

Valid Values

Values are available from a drop-down list:

tlPPMUForceIMeasureV	Force current measure voltage.
tlPPMUForceVMeasureI	Force voltage measure current.

Usage

When making a serial measurement, the pin on which you are taking the measurement uses the MeasureMode and ForceCond1 parameters. All other pins that are not being measured at that time are defined by the BackgroundMeasureMode and BackgroundForceCond parameters.

Related Topics

- [MeasureMode](#)
- [ForceCond1 and ForceCond2](#)
- [TestingInSeries](#)
- [BackgroundForceCond](#)

- [BackgroundForceIRange](#)

- [BackgroundMeasIRange](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlPinPmu.8.20

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : BackgroundForceCond

BackgroundForceCond

The voltage or current to be forced on the specified pins for serial testing.

Valid Values

Enter a value or equation in the data box. If BackgroundMeasureMode is tlPPMUForceIMeasureV, then the units are in amps; if it is tlPPMUForceVMeasureI, then the units are in volts.

Usage

See PPMU Force Current and Force Voltage Values for a list of force current and force voltage values for UltraFLEX instruments:

When making a serial measurement, the pin on which you are taking the measurement uses the MeasureMode and ForceCond1 parameters. All other pins that are not being measured at that time are defined by the BackgroundMeasureMode and BackgroundForceCond parameters.

To enter an equation, click the ellipsis button (...) to bring up the Expression Builder data box. An equation can use spec variables. The Available Specs menu lists all spec variables defined on the Specs sheets. You can also enter a numeric value or an equation in the data area.

Once you enter an equation, the present value of the equation is displayed to the right of each button display. The equation is evaluated using spec variable values determined by the current job context and DC Specs category and selector.

Related Topics

- Expression Builder
- MeasureMode
- ForceCond1 and ForceCond2

- [TestingInSeries](#)
- [BackgroundMeasureMode](#)
- [BackgroundForceRange](#)
- [BackgroundMeasRange](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tlPinPmu.8.21

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : BackgroundForcelRange

BackgroundForcelRange

The PPMU force current (I) range for serial testing.

Valid Values

Values are available from a drop-down list (no default):

AWG6G, GigaDig, HSD1000, UltraPin800, UltraPin1600	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200uA, 2mA, Auto
DCIO	8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 32mA, Auto
HPM	20uA, 200uA, 2mA, 30mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage

This parameter is valid when BackgroundMeasureMode is tlPPMUForcelMeasureV.

The Auto range makes a number of measurements to determine the best range.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).

If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- TestingInSeries
- BackgroundMeasureMode
- BackgroundForceCond
- BackgroundMeasIRange

tlPinPmu.8.22

<

>

Pin PMU VBT Test : Pin PMU VBT Test Parameters : BackgroundMeasIRange

BackgroundMeasIRange

The PPMU measure current (I) range for serial testing.

Valid Values

Values are available from a drop-down list (no default):

AWG, GigaDig	20uA, 200uA, 2mA, 50mA, Auto
BBAC, TurboAC, VHFAC	200nA, 2uA, 20uA, 200uA, 2mA, Auto
DCIO	- Non-merged PPMU Ranges: 8uA, 32uA, 128uA, 512uA, 2mA, 8mA, 64mA, Auto - Merged PPMU Ranges: 16uA, 64uA, 256uA, 1mA, 4mA, 16mA, 32mA, 128mA, 256mA, 512mA, 1.024A, 2.048A
HPM	20uA, 200uA, 2mA, 30mA, Auto
HSD1000, UltraPin800, UltraPin1600	2uA, 20uA, 200uA, 2mA, 50mA, Auto
SB6G	200uA, 2mA, 50mA, Auto
UltraPin4000	200uA, 2mA, 30mA, Auto

Usage

This parameter is valid when BackgroundMeasureMode is tIPPMUForceVMeasureI.

The Auto range makes a number of measurements to determine the best range.

When using the HPM 20A range, set the number of samples up to a maximum of 100 to ensure that test results are within tester specifications.

Available ranges in the drop-down menu depend on which instrument boards are listed in the CurrentConfig.txt file (or SimulatedConfig.txt file, if offline).

If you select a range that is not supported by instrument you are using, then IG-XL selects the next highest available range. If you select a range that is too large for the instrument you are using, then IG-XL returns a validation error.

Related Topics

- TestingInSeries
- BackgroundMeasureMode
- BackgroundForceCond
- BackgroundForceIRange

tlTopics.2.01

<

>

Creating and Editing Test Instances

Creating and Editing Test Instances

This section describes how to create test instances of the VBT tests included in the VBT test library. It contains information on how to use the IG-XL TDE Instance Editor to create test instances of the VBT tests included in the VBT test library. It also provides an overview of the VBT test use model and test structure. Finally, it describes how to create interpose functions, conduct pattern threading, and run concurrent patterns.

This section includes the following topics:

- [Creating a Test Instance](#)
- [Using an Instance Editor](#)
- [Test Instance Validation](#)
- [VBT Test Structure: Prebody, Body, Postbody](#)
- [Writing Interpose Functions](#)
- [Pattern Threading and MTD Patterns](#)

For information on instrument-specific VBT tests, see the following topics:

- [DCVI Power Supply VBT Test](#)

- [DCVS Power Supply VBT Test](#)
- [Empty VBT Test](#)
- [Functional VBT Test](#)
- [MTO Memory VBT Test](#)
- [Pin PMU VBT Test](#)

For information on parameter groups and tabs common to all of these VBT test types, see [Common Parameters](#).

<

>

2015 Teradyne, Inc. All rights reserved.

tlTopics.2.02

<	>
-----------------	-----------------

Creating and Editing Test Instances : Creating a Test Instance

Creating a Test Instance

You can create a new test instance based on an existing VBT test in two ways. The VBT test can be part of the Teradyne-supplied library or a user-created VBT test.

On a Test Instances sheet:

1. Add a row for the new test instance.
2. Enter a unique test name in the Test Name column.
3. Select VBT from the drop-down list in the Instance Type column.

The Instance Name column becomes a drop-down list containing the names of all VBT tests in the workbook.

4. Select the VBT test type from the drop-down list in the Instance Name column.

The following test types are available:

- DCVIPowerSupply_T (see DCVI Power Supply VBT Test)
- DCVSPowerSupply_T (see DCVS Power Supply VBT Test)
- Empty_T (see Empty VBT Test)

- [Functional_T](#) (see [Functional VBT Test](#))
- [MtoMemory_T](#) (see [MTO Memory VBT Test](#))
- [PinPMU_T](#) (see [Pin PMU VBT Test](#))

You can also add a test directly to a Flow Table sheet. To do this:

1. [Select the row before which you want to insert a test.](#)
2. [Right-click a cell in the row and select Insert Test.](#)

Using this method, you can add a test instance that already exists on a Test Instances sheet, add a copy of an existing test instance, add a new instance based on an existing VBT test, or add a new instance based on a VBT test that you create at this point. See [Inserting a Test in the Flow](#) in the DataTool section.

Each test instance has an automatically generated instance editor to help you enter test parameter values. For more information, see [Using an Instance Editor](#).

<

>

2015 Teradyne, Inc. All rights reserved.

tlTopics.2.03

<	>
---	---

Creating and Editing Test Instances : Using an Instance Editor

Using an Instance Editor

To help you enter parameter values for a test instance, each Teradyne-supplied VBT test has an instance editor. The instance editor lists all of the test’s parameters and provides a field for entering a parameter value.

This section is a general description of how to use an instance editor. Note that you use the same instance editor whether the instance is based on a test from the Teradyne-supplied library or on a VBT test you create. You can also use the same instance editor for test procedures you create.

The following topics are available:

- [Bringing Up an Instance Editor](#)
- [Entering Parameter Values](#)
- [Instance Editor Commands, Buttons, and Control](#)
- [Instance Editor Tools for Entering Values](#)
- [Customizing the Instance Editor](#)

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlTopics.2.04

<	>
-----------------	-----------------

Creating and Editing Test Instances : Using an Instance Editor : Bringing Up an Instance Editor

Bringing Up an Instance Editor

To bring up the instance editor to edit a specific test instance:

- On a Test Instances sheet, select the test instance name, right-click, and select Edit.
- On a Flow Table sheet, select the test instance name in the Command Parameter column. Right-click and select Edit>Instance from the menu.

There is a difference between these methods. If you open the instance editor from a Flow Table sheet, you can edit any limits values that are stored for the test instance in the Flow Table. If you open the instance editor from a Test Instances sheet, you cannot view or edit such limits values. For more information, see Limits Tab.

The instance editor comes up as a window in TDE. You can have several instance editors open at once.

The instance editor is generated each time you invoke it. It is not stored anywhere in the workbook. Once an instance editor is displayed, you can also bring up its siblings. See Viewing a Sibling Instance.

Instance Editor Information

Once an instance editor appears, it presents the following information about the instance:

- The name of the current test instance appears in the control labeled Instance:. This control's drop-down contains a list of sibling instances. See Viewing a Sibling Instance.
- The name of the VBT test or test procedure on which the test instance is based appears in the instance editor title bar.

Viewing a Sibling Instance

A sibling instance is another test instance that is based on the same VBT test. The control labelled Instance: displays the name of the current test instance. The control is also a drop-down list, which contains the names of all other instances on the Test Instances sheet that are based on the same VBT test. (The instance editor title bar contains the name of the VBT test.) Selecting a test from the list replaces the current instance with the new instance in the same instance editor window.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tlTopics.2.05

<

>

Creating and Editing Test Instances : Using an Instance Editor : Entering Parameter Values

Entering Parameter Values

The instance editor is designed to make it easy to enter parameter values. Different kinds of parameters use different forms of entering its values, as described here.

Restricted Set of Values

Some parameters have a predefined, restricted set of values, such as True and False or a set of possible ranges. These parameters have a drop-down list with the set of valid values. Select a value from the list.

Workbook Values

Some parameters take objects that are defined elsewhere in the workbook, such as the currently active timing sheets or the available VBT functions. These parameters have a drop-down list with the set of valid values. Select a value from the list. These lists always include a blank entry to indicate that you do not want a value here.

Numeric Values

Some parameters take a numeric value, either as a number or as a formula. These parameters have an ellipsis button that brings up the Expression Builder, which helps construct a formula using the variable defined on the Specs sheets. You can also directly enter a number or a formula. A formula must begin with an equal sign.

 Use the InstanceEditor>Formula command or button to toggle between displaying a formula and displaying its current value, which is evaluated using the current timing and levels context. For more information, see:

- Using Formulas
- Expression Builder

Pins and Patterns

Some parameters take a pin or pattern. The instance editor provides tools for selecting pins and patterns, invoked using the ellipsis button. You can also directly enter pin or pattern names into a field. For a list, separate the items using commas.
For more information, see:

- Pins Selection
- Pattern Selector

Strings

Some parameters take a string. Type the string directly into the field. Do not enquote it.

Additional Points on Entering Values

No checking occurs when you enter parameter values. When the test program is validated, the parameter values are also validated.
Changed parameter values appear in blue. They remain blue until you exit from the editor.
Edits you make in the instance editor are written immediately to the Test Instances sheet, where they also appear in blue. These changes are saved when you save the workbook.
You can use Edit>Undo (Ctrl+Z) to undo changes you have made to parameter values in the instance editor. You can then use Redo to reapply changes you have undone.
For more information, see [Test Instance Validation](#).

<

>

2015 Teradyne, Inc. All rights reserved.

tlTopics.2.06



Creating and Editing Test Instances : Using an Instance Editor : Instance Editor Commands, Buttons, and Control

Instance Editor Commands, Buttons, and Control

When the instance editor comes up in TDE, it has commands available in the standard TDE menus. At the top of the editor window are buttons for commonly used commands, as well as a control that does not correspond to any command.

File Menu

Close Instance Editor	Closes the currently active instance editor.
Close All Instance Editors	Closes all currently open instance editors.
Save To Copy As	Saves the current instance, with its parameter values, to a new copy. A box appears asking for the name of the new copy. The new instance is saved to the Test Instances sheet and becomes the current instance in the editor.

Edit Menu

Undo	Ctrl+Z Undo the most recent editing change.
Redo	Redo the most recent undo.
Copy	Ctrl+C Copy the selected value.

Paste	Ctrl+V
Paste the copied value into the selected field.	

InstanceEditor Menu

Lock		Toggle between locked and unlocked states. The unlocked state permits changes to the appearance of the instance editor. See Customizing the Instance Editor.
Insert Group Name		Insert a group name above the current parameter. See Customizing the Instance Editor.
Delete Group Name		Delete the currently selected group name. See Customizing the Instance Editor.
Move Down, Move Up		Move the current parameter down or up. See Customizing the Instance Editor.
Formula		Toggle between displaying numeric parameters as formulas and as evaluated values.

Instance Control

Apart from the buttons, the instance editor has a control labeled Instance:. This control displays the name of the current test instance. Its drop-down list contains the names of the test instances that are based on the same VBT test as the current instance. See [Viewing a Sibling Instance](#).

		
	2015 Teradyne, Inc. All rights reserved.	

tlTopics.2.07

<

>

Creating and Editing Test Instances : Using an Instance Editor : Instance Editor Tools for Entering Values

Instance Editor Tools for Entering Values

The instance editors provide the following tools, invoked by the ellipsis button (...) to help you enter values:

- Expression Builder
- Pins Selection
- Pattern Selector

Expression Builder

Use the Expression Builder window to build an equation using spec variables. The equation can be used as a force condition, a timing value, and other numeric parameters.
Invoke the Expression Builder window by clicking the ellipsis button (...) to the right of the data box that will receive the value. The window includes these controls:

Expression being built	This box displays the state of the equation you are building.
Available Specs	This list displays all specs currently defined on DC Specs and Global Specs sheets. Click to select a spec or to deselect one.
Add, Subtract, Multiply, Divide	Operations to join specs.
()	Parentheses that control the order of evaluation.

Clear	Clears the Equation being built window.
OK	Accepts the equation.

Once the dialog box is displayed, follow these steps:

1. Double-click the spec that should begin the equation. It is displayed in the expression box.
2. Select the next spec and click an operation (Add, Subtract, Multiply, or Divide) to join the specs. The operator and spec are appended to the equation being built.

If you have more than one spec selected, click an operator button to append all selected specs, each preceded by the selected operator. If you double-click a spec when more than one is selected, they are all appended to the equation, each preceded by a blank.

3. Add parentheses where needed to control the order of evaluation.
4. At any point, directly edit the Equation being built display to insert integer values, correct mistakes, or remove unwanted blanks. If you edit directly, note where you leave the insertion point. If you subsequently add specs or operators, they are inserted at the insertion point.
5. When you complete the equation, click OK to accept it.

 Once you apply the equation and return to the Instance Editor, you can toggle between the equation and the current value of the equation using the Formula command or button.

Pins Selection

Use the Pins Selection window to specify a pin list. Invoke the window by clicking the ellipsis button (...) to the right of the data box for the list. To use the dialog box, do the following:

- To move pins between the Available Pins and Selected Pins windows, highlight the items and use the > and < buttons.
- To move all pins from one window to the other, use the >> and << buttons.
- To move pins up or down in Selected Pins, select one or more items, and then use the ^ and v buttons at the right of the window to move them up or down.

- [When](#) Selected Pins includes the pins you want, click OK.

Pattern Selector

Use the Pattern Selector window to specify a pin list or a pattern list. Invoke the dialog box by clicking the ellipsis button to the right of the data box for the list.

The controls on the dialog box are:

Disk Patterns	Brings up a file browser to let you navigate to and select a pattern file on disk.
Pattern Groups and Sets	Brings up the Groups and Sets Selection window to let you select from pattern sets defined in the workbook. This window contains the names of all entries on the currently active Pattern Sets sheets. Note that pattern groups are not supported in UltraFLEX. • To move items between the Available and Selected windows, highlight the items and use the > and < buttons. • To move all items from one window to the other, use the >> and << buttons. • To move items up or down within the Selected field, select one or more items, and then use the ^ and v buttons at the right of the window to move them up or down. • When the Selected field displays the items you want, click OK.

If you enter multiple patterns or groups, they are separated by commas.

You can also enter a comma-separated list of patterns in the Patterns box. Any pattern file pathnames that aren't fully qualified are resolved relative to the working directory, that is, the folder containing the IG-XL workbook.

The order of the patterns in the list determines the order in which they will be executed.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlTopics.2.08

	
---	---

Creating and Editing Test Instances : Using an Instance Editor : Customizing the Instance Editor

Customizing the Instance Editor

You can change the appearance of an instance editor. This feature is intended for customizing instance editors that are generated for user-created VBT tests and test procedures.

 Note:

Although you can modify the instance editors for the Teradyne-supplied VBT test library, the changes do not persist the next time the editor is started. You cannot permanently customize these instance editors.



To customize an instance editor, it must be unlocked. Use the Lock button or

InstanceEditor>Lock to toggle between the locked and unlocked states. In the unlocked state, all fields in the Name, Value, and Comment columns are white and editable. In the locked state, only Value is white and editable; the other columns are gray and not editable.

When the instance editor is unlocked, you can perform these customizations:

- Add a group name.

A group name is the bold name that groups parameters logically and has a control to show or hide the members of the group. Select a parameter and use InstanceEditor>Insert Group Name to insert a group name before it. All the parameters from the new group name to the next group name are in the group.

Use Delete Group Name to delete the selected group name.

- Rename a parameter or group name.

Double-click the name to make it editable.

- [Change the order or parameters or group names.](#)

  [Select the item to be moved and use the buttons or InstanceEditor>Move Down or Move Up](#) command.

- [Add a comment.](#)

[Double-click the](#) Comment field to make it editable.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tlTopics.2.09

	
---	---

Creating and Editing Test Instances : Test Instance Validation

Test Instance Validation

When IG-XL validates a test program, it also performs some validation of parameter values that have been entered for the test instances.

Note that the instance editors do not perform any validation or checking when you enter data. Any test instance validation is performed only when you validate the entire test program.

When performing validation, IG-XL uses the Flow Sheet and its current context settings to determine which test instances will actually be used during test execution. In this way, it validates only those test instances that will be used. This means that some test instances on the currently active Test Instances sheet are not validated.

The following validation checks are performed.

Validation of Data Types

IG-XL checks every parameter value to ensure that the type of the specified value matches the type required by the parameter.

Validation of Patterns and Labels

IG-XL checks arguments supplied for the hold-state and preconditioning pattern. It ensures that the patterns or pattern sets actually exist. It checks that any start and stop labels exist in the pattern and are valid for use as start and stop labels. In addition, it automatically loads the patterns at this time.

If the hold-state pattern parameter PatternTimeout is specified, IG-XL ensures that the argument is a number greater than 0.

Validation of Pin State Values

IG-XL verifies that any pins specified for these parameters are appropriate for their intended use. Parameter arguments are checked to verify that they do not overlap, that is, that they do not contain the same pins. Pins are also checked for appropriate pin types in the pin map. For example, pins specified for Util0Pins and Util1Pins must be utility pins.

Validation of Interpose Functions

If an interpose function parameter is specified, IG-XL verifies that the function exists.

Validation of Test-Specific Parameters

Most of the validation described in this topic relates to common parameters that are used by several or all tests in the library: for example, pin states, hold-state patterns, and interpose functions. Any additional validation of test-specific parameters depends on the individual VBT test in the Teradyne-supplied library. For example, the library test could check whether a certain parameter has a nonblank value or whether its value is within a certain range.

To see what validation is performed for a particular test type, you can look at the code:

- | | |
|----|--|
| 1. | Bring up the VB Editor using Alt+F11. |
| 2. | In the Project Explorer, open the Template (Template.xla) project. |
| 3. | Under Modules, click the module for the test type you want to see. The test modules begin with VBT_ and end with _T: for example, VBT_MtoMemory_T. |
| 4. | Each test module has a Private Function called Validate. Examine this function to see what validation is performed for the specific library test. |

Note that all parameter values, both common and test-specific, are always checked for the correct data type.

tlTopics.2.10

<

>

Creating and Editing Test Instances : VBT Test Structure: Prebody, Body, Postbody

VBT Test Structure: Prebody, Body, Postbody

VBT tests in the test library all have the same basic structure, which includes:

Prebody	Prepares for testing. Activities include: obtaining levels and timing information for any patterns to be run, computing the number of samples to be made, and setting initial states on pins.
Body	Performs the testing. Typical Body activities include: running any patterns for preconditioning or dynamic conditioning, connecting and disconnecting specified pins, performing a functional test or parametric measurement, and running any user-written interpose functions at the appropriate points.
Postbody	Performs any post-test clean-up functions. Typical Postbody activities include restoring pin electronics to their entry state and returning channels to their default start-state conditions.

This structure makes these tests “steppable.” In debug mode, you can step over each part of a test separately, pausing after the Prebody and the Body. In regular execution mode, the test is executed as a single step.

The major advantage of the three-part structure is that it separates one-time activities (setup and clean-up) from the activities that may be repeated in a loop (actual testing). In this way, you can construct a test, such as a shmoo test, that loops over the Body, while executing the Prebody and Postbody only once.

Some interpose functions are defined by their relationship to this three-part structure. See [Interpose Functions Parameters](#).

In debug mode, flow arrows indicate the current step of a steppable test. For the arrows on the DataTool Flow Table sheet, see [Step, Step Over, and Step Out](#) in the DataTool section. For the arrows in MiniFlow, see [Debugging Using MiniFlow](#) in the TDE section.

User-created VBT tests can also be written as “steppable” tests, following the model of the VBT library tests. See [Creating Steppable Tests](#) in the VBT Syntax Reference.

Tests based on user-created custom templates under previous releases of IG-XL are also steppable. These tests are identified in the Instance Type column as Template. Note that the VBT tests in the current release are of type VBT, and are not the same as the previous templates, which are no longer supported. The Template type is retained in the current release so that older tests based on user-created templates can continue to execute.

<

>

2015 Teradyne, Inc. All rights reserved.

tlTopics.2.11

<

>

Creating and Editing Test Instances : Writing Interpose Functions

Writing Interpose Functions

This section includes the following topics:

- [Adding Interpose Functions](#)
- [Function Names](#)
- [Calling Convention](#)
- [Examples](#)
- [Accessing Spec Values](#)

This section describes only the basic format of an interpose function, not the details of writing VBT code. For more information on VBT syntax, see the [VBT Syntax Reference](#).

Adding Interpose Functions

Add interpose functions to the test program workbook. Interpose functions must be located in the “Modules” component of the VBAProject project for the workbook.

1. [With the workbook open, invoke Visual Basic: Alt+F11 or Tools>Macro>Visual Basic Editor.](#)
2. [In the Project Explorer, select “VBAProject \(workbook.xls\).”](#)
3. [Use the Insert>Module command.](#)

The new module is inserted into the Modules folder. (You can rename the module in the Properties window. See the note on names in the next paragraph.) Create the interpose functions in this module.

Function Names

The name of an interpose function cannot be the same as the name of the module containing the function. If the names are the same, a run-time error is returned.

Calling Convention

The calling convention for an interpose function is:

Public Function name(argc As Long, argv() As String) As Long

An interpose function should have two arguments:

argc	A Long indicating the number of arguments passed to the function.
argv	An array of strings, one element for each argument passed.

The function can use argc and argv to process the arguments.

The instance editor allows you to enter a comma-delimited list of parameters for each interpose function. When a test instance calls an interpose function, the list is automatically translated into the argc, argv format and passed to the function.

Follow this calling convention. For interpose function parameters, the instance editor displays a list of all functions in the workbook that follow the calling convention. This means that you can select a function name from the list rather than having to enter the name.

An interpose function returns a value of type Long. A zero (0) value means no error. Any nonzero value means that an error has occurred. By default, an interpose function returns a zero. The function can explicitly set a return value. The return value is passed back through to the caller. VBT tests in the test library respond to the return value as follows:

- If any pattern interpose function (PrePat, PostPat, PreTest, and PostTest) returns a nonzero value, IG-XL interprets this as an error and will bin out to the error bin, if set.
- Values returned by other, nonpattern interpose functions are ignored.

Examples

The following is a simple interpose function that sets the global address register with a pattern name and a label in the pattern. The PatGen method SetGlobalAddr takes a pattern name and a label as its parameters.

The first example has the pattern and label names hard-coded:

```
Public Function PatGlobal(argc As Long, argv() As String) As Long
    Dim RtnSt
    RtnSt = TheHdw.Digital.PatGen.SetGlobalAddr("lz0pat","rz_start")
End Function
```

For this interpose function, the instance editor would specify the function name and no parameters. You can replace the hard-coded values to make the function more general. The argument count is used to determine if the label was included; if it is not, a standard label name is used:

```
Public Function PatGlobal(argc As Long, argv() As String) As Long
    Dim RtnSt
    If argc = 2 Then
        RtnSt = TheHdw.Digital.PatGen.SetGlobalAddr(argv(0),argv(1))
    Else
        If argc = 1 Then
            RtnSt = TheHdw.Digital.PatGen.SetGlobalAddr(argv(0),"rz_start")
        End If
    End If
End Function
```

Accessing Spec Values

Frequently, an interpose function called from a VBT test must access the spec values that are used in the test. You can use the TheExec.Specs object to access spec values. See Specs in the VBT Syntax Reference.

tlTopics.2.12

<

>

Creating and Editing Test Instances : Pattern Threading and MTD Patterns

Pattern Threading and MTD Patterns

When executing a pattern at multiple sites, IG-XL presents the programming model of a single pattern generator (PatGen) running the pattern at all sites and instruments at the same time in lockstep. In fact, multiple PatGens are distributed throughout the test system. IG-XL coordinates them, grouping them and running them in lockstep, thereby maintaining the model of a single PatGen.

IG-XL supports several models for executing patterns that involve multiple sites and instruments. The kinds of PatGen execution are:

Shared PatGen	A single PatGen can be shared by multiple sites. All sites must be executing the same pattern vectors at the same time. Default.
Pattern threading	Each site has its own PatGen, so that each site's pattern executes independently of other sites and can make its own branching decisions. This is also known as "PatGen per site."
MTD patterns	Patterns in multiple time domains are started at the same time. Each pattern or set of PatGens executes independently of other patterns and can make its own branching decisions. MTD patterns allow you to run both digital and analog patterns at the same time.

The Functional Test and MTO Memory Test both have a Threading argument, which turns pattern threading on and off.

The following topics are available:

- Pattern Threading

•	MTD Patterns
---	------------------------------

 Note:

Using pattern threading and MTD patterns at the same time is only supported when using digital instruments, such as DCIO, HSD1000, SB6G, and UltraPin800. When pattern threading is on, all sites must start at the same time and run the same pattern. You cannot start a new pattern if any site is running the old pattern. Pattern threading does not support analog instruments.

			
		2015 Teradyne, Inc. All rights reserved.	

tlTopics.2.13

<	>
--------------------------------------	--------------------------------------

Creating and Editing Test Instances : Pattern Threading and MTD Patterns : Pattern Threading

Pattern Threading

By default, with pattern threading turned off, a PatGen is shared by several sites, and all sites must be executing the same pattern vectors at the same time.

For most testing, this restriction is acceptable. If a pattern uses match loops, however, this restriction would require the patterns to be executed serially, thereby reducing throughput. In this case, each site should make its own branching decisions, independent of other sites. Pattern threading permits independent execution of the match loops on each site.

If pattern threading is turned on, it applies to all patterns in a test instance. A job, however, can contain instances that do and do not use pattern threading.

Even when pattern threading is turned on, all sites must start at the same time and must run the same pattern. A new pattern cannot be started if any site is still running the old pattern.

The Functional Test and MTO Memory Test both have a Threading argument, which turns pattern threading on and off.

Validation

When pattern threading is on, IG-XL performs additional validation checks:

- | | |
|--|--|
| | <ul style="list-style-type: none">• No PatGen hardware module (channel board or instrument) containing a PatGen can be mapped to more than one site. |
| | <ul style="list-style-type: none">• The number of sites in the channel map cannot exceed the maximum number of digital instrument boards in the test system. |

Effects on Interpose and PatFlag Functions

If pattern threading is turned on, it affects execution of PostPat and PatFlagFunc interpose functions, which are called after pattern execution.

The PostPatF interpose function is called after a pattern has halted. If pattern threading is turned on, sites may halt at different times. The PostPatF interpose function is called once, after all sites have halted. The interpose function can also use a serial loop to check the results of each site and respond appropriately on a per-site basis.

The PatFlagF interpose function is called during pattern execution when the PatGen’s CPU flags match a specified Wait Flags setup. If pattern threading is turned on, the sites may match at different times, and the interpose function can be called when any one site matches or when all sites match. The argument MatchAllSites lets you select whether one site or all sites must match. If MatchAllSites is Yes, the PatFlagF function is called only when all sites match; if it is No (the default), it is called when the first site matches. See [Flag Match Parameters](#).

<

>

2015 Teradyne, Inc. All rights reserved.

tlTopics.2.14

	
---	---

Creating and Editing Test Instances : Pattern Threading and MTD Patterns : MTD Patterns

MTD Patterns

By default, pattern execution in multiple time domains is not used. When a single pattern is started, all instruments in the pattern execute the same vector and PatGen commands at the same time. As with pattern threading, this single pattern execution is acceptable for most tests where synchronization is a desirable if not an absolute requirement for proper functionality.

But some devices have modules (cores) that can operate independently of each other. IG-XL supports executing multiple patterns at the same time, each pattern testing a different module (core) of the DUT. The patterns contain the instruments, time sets, vector data, and PatGen commands necessary to test the target module.

MTD pattern execution saves test time when compared to serial testing of modules. It can also provide a test environment that better matches the end application.

Validation

When MTD patterns are used, IG-XL performs an additional validation check. No PatGen hardware module (channel board or instrument) containing a PatGen can be mapped to more than one of the concurrent patterns being executed at the same time.

Timing and Levels Context

MTD patterns share the same timing and level context, but the patterns can use different time sets from a timing sheet. For example, pattern1 can use time sets tset_1 and tset_2, while pattern2 uses time sets tset_3 and tset_4. The time sets can be defined on the same Time Sets or Time Sets (Basic) sheet or on different sheets.

Functional Test Instance

Define multiple patterns to be executed at the same time in the Patterns input argument using a plus sign, +. For example to execute "pat1" and "pat2" at the same time, you would set the input argument to "pat1+pat2". The list can be include individual patterns. Pattern sets and pattern groups are not supported.

VBT

The pattern list passed to VBT commands that execute patterns can use the plus sign to execute patterns at the same time, with the same syntax as the Function Test Instance.

Effects On Interpose and PatFlag Functions

If MTD patterns are executed, this affects execution of PostPat and PatFlagFunc interpose functions.

The PostPatF interpose function is called after a pattern has halted. If MTD patterns are being executed, the patterns will halt at different times. The PostPatF interpose function is called once, after all patterns have halted.

The PatFlagF interpose function is called during pattern execution when the PatGen’s CPU flags match a specified Wait Flags setup. If concurrent patterns are being executed, the patterns may match at different times, and the interpose function can be called when any one pattern matches. You must determine which pattern met the match condition and handle any pattern specific requirements. See [Flag Match Parameters](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	